

Архітектура системи команд (ISA) x86



©Олег Фаренюк,
Creative Commons - Attribution Share Alike (CC BY-SA)

ISA – Instruction Set Architecture

- Instruction Set Architecture – Архітектура системи команд, абстрактна модель CPU.
- ◇ Описує виконання машинного коду цим CPU.
- ◇ Може мати багато реалізацій, які відрізняються швидкодією, ціною тощо.
- ISA є видимою програмістам частиною CPU – інтерфейсом між програмним та апаратним забезпеченням.
- Вона включає:
 - ◇ Машинні команди (instruction)¹,
 - ◇ Регістри,
 - ◇ Типи даних,
 - ◇ Способи адресації пам'яті,
 - ◇ Моделі узгодженості пам'яті, способи керування віртуальною пам'яттю та керування особливостями мікроархітектури² тощо.Вони будуть розглядатися в наступних курсах.

¹Мова про машинне представлення, але ми більше матимемо справу із мнемоніками асемблера, які їм відповідають, та способом їх кодування.

²Знаходяться на межі між ISA та мікроархітектурою.

x86

- ISA x86 має довгу історію, тому є складною і плутаною.
- Зокрема, вона існує в 16-бітовому, 32-бітовому та 64-бітовому варіантах, кожен із яких підтримує всі попередні версії.
- Ми будемо розглядати 64-бітовий варіант, іноді з врахуванням 32-бітового.
- ◇ Обидва вони використовуються сучасними ОС³.
- Детально будемо знайомитися із user mode командами для цілочисельної арифметики.
- ◇ Підтримка floating-point розглядається коротко.
- Із kernel mode розглядаються лише концепції, типу обробки переривань.

³Більшість сучасних 64-бітових ОС підтримують 32-бітові програми "з коробки" чи з встановленням додаткових програмних пакетів.

Програма на асемблері

- Асемблерна програма містить, в цілому, знайомі елементи.
- Директиви, які керують генерацією коду (наприклад – для якої архітектури чи її варіанту, асемблювати; до якого сегменту відносити конкретні частини програми тощо).
- ◇ За допомогою директив оголошуються змінні, символи чи константи – сутності, що мають ім'я і значення, але не зберігаються в пам'яті.
- ◇ Також, багато асемблерів підтримують макроси.
- **Мнемоніки** машинних команд (instructions), які слід виконати,
- ◇ разом із їх **операндами** – аргументами, над якими команди працюватимуть.

Приклад коду Linux-64, мінімалістичний, GNU as, Intel syntax

```
.intel_syntax noprefix # Same as: -msyntax=intel -mnaked-reg
.global _start

.text
_start:
    mov eax, 1      # __NR_write (sys_write) == 1
    mov edi, 1      # Zero extension -- верхня частина RAX, RDI і тд.. зануляється
    mov rsi, OFFSET msg # Non-PIC код, подробиці далі
    mov edx, OFFSET msg_size
    syscall
    mov eax, 60      # __NR_exit (sys_exit) == 0
    xor edi, edi     # exit code 0
    syscall

.data
msg:
.ascii "Hello 64-bit Linux world!\n"
msg_size = . - msg
```

- Містить директиви GNU as⁴: .text, .data, .ascii, тощо – розпочинаються з крапки. "Самотня" крапка – адреса поточного місця в пам'яті, OFFSET – адреса змінної (або значення символу, e.g. msg_size); та мнемоніки команд – mov, syscall тощо⁵.

⁴Сильно відрізняються від асемблера до асемблера.

⁵Вони менш-більш однакові у різних асемблерів – оскільки визначаються ISA 

Особливості GNU as

- В світі асемблерів x86, GNU as (GAS) – доволі нетиповий⁶⁷, порушує багато конвенцій.
- Однак, для інших платформ – ARM, RISC-V, SPARC, MIPS тощо, він, хоч і не завжди єдиний, але часто – розумний вибір.
- ◇ Крім того, зручно використовувати єдиний асемблер для різних платформ – щоб не відволікатися на особливості директив.
- Тому, після пів-десятиліття вагань, автор курсу вирішив перейти на GAS⁸.
- GNU as використовує, за замовчуванням, асемблер AT&T.
- ◇ Про нього далі, але, він, наприклад потребує сигілів (sigil) – спеціальних символів перед константами (\$msg) та інших (%RAX)– перед іменами регістрів.
- ◇ Операнди інструкцій **переставлені** відносно порядку, прийнятого для процесорів Intel: "mov eax, 5" проти "mov \$5, %eax".
- ◇ Розмір операндів задається в імені інструкції (movl ...) на противагу підказкам для Intel в неоднозначних ситуаціях (mov DWORD PTR ...)
- В цьому курсі використовуємо **GAS у режимі "емуляції" синтаксису Intel** і без використання sigil для регістрів. Однак, відмінності залишаються – див. далі.

⁶На відміну від більшості інших ISA, для цієї і її "родини" – Z80, 8080, існує безліч асемблерів. Лише із сучасних, відкидаючи виключно 16-бітових, типу (класного!) TASM чи дивні речі типу HLA (High level assembler), зразу можна згадати: NASM, MASM, FASM, JWASM, YASM, JWASM, WASM. То всі вони між собою помітно більш схожі, ніж вони з GNU as.

⁷Щодо асемблерів для 8-бітових процесорів "метасімейства", див.
https://indrekis.github.io/retrocomputing/cpm/cp/m/2023/07/18/asm_1.html.

⁸Хоч і почуває себе трохи зрадником. Також, ідіоми інших асемблерів можуть все ще траплятися в прикладах.

Особливості GNU as в режимі синтаксису Intel

- Оголошення стрічкових констант інших асемблерів традиційно виглядає якимось так:

```
msg db 'Hello 32-bit Linux world!',0xA
```

- Директиви керування сегментами, в цілому (історично) більш розвинуті:

```
segment readable executable ; Аналог .text,  
; але i GAS має: .section "name"[, flags...]
```

- Символом початку коментаря, як у прикладі вище, часто служить крапка з комою.
- Часто присутні розвинуті системи макросів та макроси типу:

```
invoke MessageBox,HWND_DESKTOP,"Hi! I'm the example program!",\n\n"Win32 Assembly",MB_OK
```

для виклику функцій з правильними угодами викликів (див. далі).

- Інші асемблери часто вміють створювати виконавчі файли безпосередньо, без явного лінування – принаймні, у простих випадках.
- Розширення GAS – .s, інші асемблери часто використовують .asm.
- Ще однією характерною особливістю GAS є лапідарні повідомлення про помилки.
- ◇ Базово це був не так асемблер для використання людьми, як бекенд для GCC.
- Шістнадцяткові та двійкові числа GAS записує в стилі C: 0xABCD, 0b1010; традиційні асемблери частіше використовують суфікси: 0ABCDh, 1010b.
- Документація: <https://sourceware.org/binutils/docs-2.40/as.pdf>.
- <https://stackoverflow.com/a/39360636> – подробиці про поведінку GAS в режимі Intel.

Особливості GNU as щодо вказання адрес

- Навіть за використання синтаксису Intel, для GAS все це синоніми⁹:

```
mov esi, [msg]          # 1
mov esi, msg            # 2
mov esi, DWORD PTR msg # 3
mov esi, DWORD PTR [msg] # 4
```

Інші асемблери зазвичай приймають 1 і/або 4, іноді також без PTR¹⁰.

- ◇ Найбільш неочікуваним є приклад 2 (і, менше, 3) – для інших асемблерів він означатиме використати адресу символу чи значення константи абощо.
- ◇ Для GAS робимо так:

```
mov rsi, OFFSET msg
mov edx, OFFSET msg_size
```

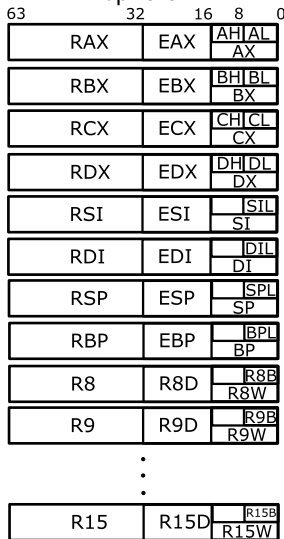
Другий випадок – особливо контрінтуїтивний для користувачів "традиційних" асемблерів x86.

⁹І автор мусить зізнатися, не розуміє, що значить ще одна можлива комбінація: "mov esi, DWORD [msg]", яка спричиняє виникнення адреси OFFSET msg + 4..., та +8, якщо написати QWORD...

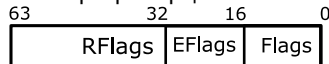
¹⁰Що для GAS означає щось зовсім інше...

Регістри x86

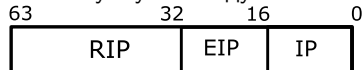
Цілочисельні регістри загального використання:



- Регістр прапорців:



- Instruction pointer, вказівник на наступну команду:



Сегментні регістри^a:

- CS, DS, ES, SS – в 64-бітовому режимі ігноруються.
- FS, GS – використовуються для доступу до thread local data (детальніше – на АКС).

^a16-бітові, містять індекс у так званій таблиці сегментів. Сегменти – застарілий спосіб керування віртуальною пам'яттю, legacy.

Регістр прапорців EFLAGS

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13,12	11	10	9	8	7	6	5	4	3	2	1	0
										I	I	V	A	V	R		N		O	D	I	T	S	Z		A		P		C
0	0	0	0	0	0	0	0	0	0	D	V	I	C	M	F	0	T	IOPL	F	F	F	F	F	F	0	F	0	F	1	F
										x	x	x	x	x	x			x x	s	c	x	x	s	s		s		s		s

- Цей регістр містить інформацію про стан процесора та деякі характеристики результату останньої операції. Також він дозволяє керувати деякими особливостями роботи процесора.
- x – системний прапорець, s – прапорець стану, c – прапорець керування, 0 – зарезервовано, завжди нуль, 1 – зарезервовано, завжди одиниця.
- Нижче розглянуто необхідні для повсякденного (не системного) програмування.
- Для покращення сумісності програми з різними варіантами процесорів забороняється пробувати змінити або покладатися на значення зарезервованих бітів!
- IOPL (Input/Output Privilege Level) – рівень привілеїв вводу-виводу. В сучасних ОС вам заборонено звертатися до портів, це поле завжди містить 00b.

- **OF** (Overflow Flag) – прапорець переповнення, встановлюється, якщо результат операції не поміщається в отримувач.
- **DF** (Direction Flag) – прапорець керування напрямком в рядкових операціях. Якщо він рівний нулю, то регістри автоматично збільшуються на кількість байт операнда, а якщо одиниці, то зменшуються.
- **SF** (Sign Flag) – прапорець знаку, дорівнює останньому (найстаршому) біту результату, є ознакою від'ємного числа при операціях над знаковими числами.
- **ZF** (Zero Flag) – прапорець нуля, дорівнює одиниці, якщо результат рівний нулю.
- **AF** (Auxiliary Flag) – прапорець допоміжного перенесення. Він служить для роботи з так званими BCD числами. (Binary Coded Decimals). Встановлюється за наявності перенесення з 3-го в 4-й біт або запозичення з 4-го¹¹.
- **PF** (Parity Flag) – прапорець парності¹², дорівнює 1, якщо в результаті операції отримано парну кількість одиниць, інакше – нуль.
- **CF** (Carry Flag) – прапорець перенесення з (або позичання в) старшого біта результату в арифметичних операціях.

¹¹BCD команди відсутні в 64-бітовому режимі.

¹²Кажуть, є залишок того, що x86 – далекий нащадок CTC Datapoint, який був терміналом – і мав підтримувати комунікацію UART-подібними протоколами, для яких зручно мати обчислення парності.

Прапорці в інших ISA

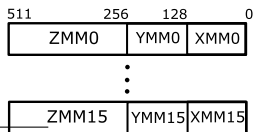
- Значна частина ISA мають прапорці, схожі на прапорці x86.
- ◇ Винятковим¹³ є прапорець парності PF – спадок DataPoint 2200.
- ◇ AF "останнім часом" теж мало-популярний, але більш поширений, ніж PF – завдяки успадкованим ISA¹⁴.
- ISA, такі як ARM, встановлюють прапорці лише, якщо про це попросити явно – відповідним опкодом.
- Інші (RISC-V, MIPS, Alpha) – не мають регістру прапорців, використовуючи для результатів операцій порівняння регістри загального використання.

¹³Ну, майже: Motorola 6809...

¹⁴Окрім сімейства x86/8080/Z80, є також в AVR8 та Motorola 6800.

ZMM/YMM/XMM-реєстри

- Сучасні процесори Intel для роботи із floating point рекомендують використовувати **SIMD-реєстри**.
- Це, фактично, невеликі масиви фіксованого розміру¹⁵.
- Вони призначені для обробки кількох значень однією машинною командою¹⁶.
- Їх нульовий елемент можна використовувати і для *скалярних* обчислень – “звичайних” операцій з floating-point числами.
- XMM – 128-бітові реєстри, впроваджені як частина SSE¹⁷
- YMM – їх розширення до 256 біт у AVX, також – кількість збільшилася з 8 до 16.
- ZMM – розширення YMM до 512 біт у AVX-512.
- Розширення AVX-512 присутнє, в основному, в серверних процесорах.

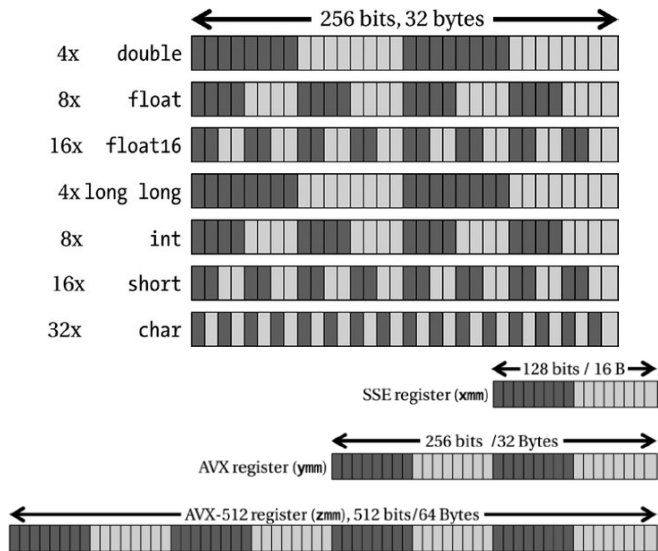


¹⁵Є ISA, де їх розміром можна, в межах наявних ресурсів, керувати.

¹⁶SIMD означає “Single instruction – multiple data”, детально ми це поняття будемо вивчати на АКС.

¹⁷Це приклад **розширення ISA** – доповнення до базової ISA у певних версіях процесорів.

- В залежності від інтерпретації програмістом та використаних команд, кожен із YMM регістрів може представляти декілька величин різного типу:



x87 FPU

- Legacy-підтримка floating point – FPU x87.
- ◇ Це так званий співпроцесор¹⁸ (coprocessor).
- Має вісім 80-бітових регістрів, організованих як стек¹⁹.
- Вершина стеку називається ST0, регістр під ним - ST1 і т.д.
- ◇ Коли в цей стек додається нове значення чи забирається старе, всі номери зсуваються.
- Хоча Intel не рекомендує, іноді вони все ще використовуються у скомпільованому коді, особливо – 32-бітовому, на додачу до YMM регістрів.
- Floating-point значення в регістрах STx завжди зберігаються у 80-бітовому форматі, який GCC для x86 представляє як long double²⁰, але при операціях із пам'яттю може завантажуватися та зберігатися як 80-бітовий, 64-бітовий (double) чи 32-бітовий float (у форматі IEEE 754, про який далі).

¹⁸Колись це була окрема мікросхема – 8087, 80287, 80387 та кілька екзотичніших, тому збірно їх називають x87. Починаючи з 486 – став вбудованим, однак назва, вже тепер – розширення ISA, збереглася.

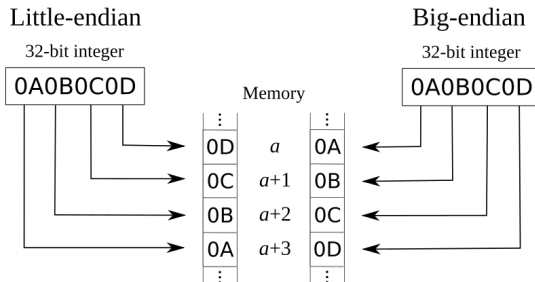
¹⁹Ці ж регістри використовуються застарілим SIMD-розширенням MMX.

²⁰Сучасний MSVC не надає доступу до цього засобу, у нього long double – синонім double.

- Додатково, x87 має службові регістри:
- ◇ x87 FPU Status Register – 16-бітовий регістр прапорців для цього співпроцесора. Повідомляє про ділення на нуль, руйнування стеку, переповнення, денормалізацію і т.д.; код вершини стеку; та містить результати порівняння двох чисел – condition code flags.
- ◇ x87 FPU Control Word – 16-бітовий регістр, що керує роботою співпроцесора: режимом заокруглення, точністю внутрішніх розрахунків (80, 64, 32 біти) та умов, які помилки повинні генерувати exception.
- ◇ x87 FPU Tag Word – 16-бітовий регістр, де кожних два біти повідомляють стан відповідного регістра даних – містить звичайне значення; нуль; містить NaN, безмежність чи денормалізоване число; порожній.
- ◇ Також, для обробки exception, містить (last) Instruction і Data (Operand) Pointers – на останню не керувальну команду, що опрацьовувалася, і на останні дані, які використовував співпроцесор, та регістр опкоду останньої команди.
- Деякі подробиці див.
https://indrekis.github.io/comp_eng/pok/2023/09/23/floating_point_1.html
керування-для-x86x87–низькорівневе і посилання там.

Endianness

- x86 – little endian.
- Тобто, байти типів даних, підтримуваних процесором, йдуть в пам'яті у зворотному порядку:



- Прийнятий для мережевих протоколів TCP/IP порядок – big endian, прямий.
- Детальніше див. <https://en.wikipedia.org/wiki/Endianness>.
- Команда x86 BSWAP змінює порядок байт на зворотній.
- Колись більшість інших ISA були big endian, однак, через популярність x86, зараз домінує little endian. Процесори типу ARM, IBM Power, Sparc, існують в обох варіантах.

Endianness – демонстрація

- Код:

```
uint32_t x = 0x12'AB'CD'EF;  
unsigned char* ptr = reinterpret_cast<unsigned char*>(&x);  
std::cout << std::uppercase << std::hex;  
for(int i = 0; i<sizeof(x); ++i){  
    std::cout << static_cast<uint32_t>(*(ptr+i)) << " ";  
}
```

- ◇ на little endian системі виведе:

EF CD AB 12

- ◇ а на big endian:

12 AB CD EF

Endianness – історія

- Детальніше про endianness див <https://en.wikipedia.org/wiki/Endianness>. Цікавинка – назва відсилається до "Подорожей Гулівера".
- Причина, цитуючи <https://www.techspot.com/article/1397-intel-8008-microprocessor/>:
"Nonetheless, some characteristics of the Datapoint 2200 still exist in today's x86 processors. For instance, the Datapoint 2200 had a serial processor, processing bytes one bit at a time. Since the lowest bit needs to be processed first, the Datapoint 2200 was little-endian. For compatibility, the 8008 was little-endian, and this is still the case in Intel's processors. Another feature of the Datapoint 2200 was the parity flag, since parity calculation was important for a terminal's communication. The parity flag has continued to the x86 architecture."
- ◊ Окрім того, що мікроархітектура була однобітною, в ролі оперативної пам'яті ті процесори використовували зсувні регістри²¹ – біти доводилося опрацьовувати в тому порядку, в якому вони "випихаються" з зсувного регістра, а, скажімо, додавання слід виконувати, починаючи із молодших бітів. Джерело: http://archive.computerhistory.org/resources/text/Oral_History/Poor_Victor/102658337.05.01.pdf – "Oral History of Victor (Vic) Poor", інтерв'ю із розробником з Datapoint.

²¹Тоді це було суттєво дешевше

Адресація пам'яті – x86-64

$$\left\{ \left\{ \begin{matrix} \text{FS :} \\ \text{GS :} \end{matrix} \right\} \begin{bmatrix} \text{RAX} \\ \dots \\ \text{R15} \end{bmatrix} + \begin{bmatrix} \left\{ \begin{matrix} \text{RAX} \\ \dots \\ \text{R15} \end{matrix} \right\} * \begin{bmatrix} 1 \\ 2 \\ 4 \\ 8 \end{bmatrix} \end{bmatrix} \right\} + [\text{displacement}] \\ [\text{RIP}]$$

- X86-64 має багато способів адресації.
- **Displacement** – в команді безпосередньо вказано зміщення:

```
mov EAX, [0x123450]
```

- ◇ В 64-бітовому режимі зміщення може бути 32-бітовим або 64-бітовим – одній мнемоніці будуть відповідати різні опкоди (див. наступний слайд).
- ◇ Таким способом компілятор може звертатися, наприклад, до глобальної змінної: <https://godbolt.org/z/c4xWr4Pov>²².
- **Base** – адреса береться з регістра:

```
mov EAX, [RSI]
```

- ◇ Використовується для звертання за вказівником, приклад: <https://godbolt.org/z/q7exo1lKK>.

²²Movabs – варіант мнемоніки mov із 64-бітовою адресою, яку використовують інструменти GNU. ▶ ◀ ≡ ≡ ≡ ↺ 🔍 ↻

Відступ: що значить, коли одній мнемоніці відповідають різні опкоди

- В **64-бітовому режимі** зміщення в `mov EAX, [offset]` може бути 32-бітовим або 64-бітовим – одній мнемоніці будуть відповідати різні опкоди²³:

```
mov EAX, [0x123456789ABC] ;      A1 BC 9A 78 56 34 12 00 00
mov EAX,      [0x56789ABC] ;    67 A1 BC 9A 78 56
; однак, GAS генерує гірший альтернативний опкод тієї ж дії:
; 8B 04 25 BC 9A 78 56
```

- Забігаючи наперед, проаналізуємо двійковий код команд:
 - ◇ A1 – опкод команди перемістити в EAX вміст пам'яті за абсолютною адресою.
 - ◇ Після неї йде, власне, адреса, в little endian.
 - ◇ 67 перед другою A1 – це так званий **префікс**, який вказує вважати адресу 32-бітовою – замість 64-бітової за замовчуванням.
- "Гірший" опкод – мається на увазі, з точки зору розміру. Можливо, GAS вважає його ефективнішим щодо часу виконання. Або просто недогляд.

²³Синтаксис FASM, [qword ...] вказує використати безпосереднє 64-бітне зміщення, [dword ...] – 32-бітове. Без цього, по замовчуванню, FASM [off] трансліює в [RIP + off], детальніше див.
<https://flatassembler.net/docs.php?article=manual#2.1.19>.

Відступ: використання 64-бітових регістрів

- Розглянемо команди, що використовують 64-бітовий регістр:

```
mov RAX, [0x123456789ABC] ;      48 A1 BC 9A 78 56 34 12 00 00
mov RAX,    [0x56789ABC] ;      67 48 A1 BC 9A 78 56
; однак, GAS генерує гірший альтернативний опкод тієї ж дії:
; 48 8B 04 25 BC 9A 78 56
```

- З'явився ще один префікс, 0x48 – він вказує використовувати 64-бітовий RAX замість 32-бітового EAX.
- Ремарка: префікси це складна тема, тут названо їх роль саме в цьому коді.
- Висновок: інженери AMD, розробляючи x86-64, обрали по замовчуванню використовувати 64-бітові адреси, але 32-бітові цілі числа.
- Це відображено у реалізаціях компіляторів для них: – sizeof(int) == 4, sizeof(void*) == 8 (байт).
- Для використання 16-бітових регістрів є свій префікс, 0x66:

```
mov AX, [0x123456789ABC] ;      66 A1 BC 9A 78 56 34 12 00 00
mov AX,    [0x56789ABC] ;      67 66 A1 BC 9A 78 56
; однак, GAS генерує гірший альтернативний опкод тієї ж дії:
; 66 8B 04 25 BC 9A 78 56
```

- А ось робота з 8-бітовими регістрами використовує інші опкоди.

$$\left\{ \left\{ \begin{matrix} \text{FS :} \\ \text{GS :} \end{matrix} \right\} \begin{bmatrix} \text{RAX} \\ \dots \\ \text{R15} \end{bmatrix} + \left[\begin{bmatrix} \text{RAX} \\ \dots \\ \text{R15} \end{bmatrix} * \begin{bmatrix} 1 \\ 2 \\ 4 \\ 8 \end{bmatrix} \right] \right\} + [\text{displacement}]$$

[RIP]

- **Base + Displacement** – адреса береться із регістру, до якого додається (знакове) зміщення:

```
mov EAX, [RSP + 0x10]
```

- ◇ В 64-бітному режимі зміщення може бути 8-бітовим або 32-бітовим²⁴. Зауважте – 64-бітовим не може бути.
- ◇ Використовується для доступу до членів структур.
- ◇ Забігаючи наперед, для функцій із типовим кадром стеку, використовується для звертання до локальних змінних та аргументів, переданих через стек. З іншого боку, стек – це теж така структура.
- ◇ Приклад: <https://godbolt.org/z/WjEfcjFTP>.

²⁴Команду з прикладу можна представити як 8B 45 10, але і як "8B 85 10 00 00 00" – вся надія на асемблер, щоб він обрав правильний опкод.

$$\left\{ \begin{array}{l} \left\{ \begin{array}{l} \text{FS :} \\ \text{GS :} \end{array} \right\} \left[\begin{array}{c} \text{RAX} \\ \dots \\ \text{R15} \end{array} \right] + \left[\begin{array}{c} \text{RAX} \\ \dots \\ \text{R15} \end{array} \right] * \left\{ \begin{array}{c} 1 \\ 2 \\ 4 \\ 8 \end{array} \right\} \end{array} \right\} + [\text{displacement}]$$

[RIP]

- **Base + (Index * Scale) + Displacement**²⁵.
- Адреса береться із базового регістру, до нього додається вміст індексного регістру, помножений на 1, 2, 4 або 8, до результату додається зміщення (8 або 32 біти).

```
mov EAX, [RBX + 2*RSI + 0x10]
```

- ◇ Може використовуватися компілятором²⁶, до прикладу, для доступу до члена структури, що знаходиться в масиві, або до глобальних двовимірних масивів:
<https://godbolt.org/z/qbfqMa56f>.
- Як окремі режими, на рівні опкодів, існує **Base + (Index * Scale)**, компілятори його люблять помітно більше за попередній; і **(Index * Scale) + Displacement**: <https://godbolt.org/z/5eWdEYYh5>.

²⁵Формально, документація окремо виділяє **Base + Index + Displacement** – із Scale==1 – ще одна аномалія x86 ISA.

²⁶Хоча й не дуже охоче – сходу лише Clang вдалося переконати.

$$\left\{ \begin{matrix} \{ \text{FS} : \\ \text{GS} : \} \end{matrix} \begin{bmatrix} \text{RAX} \\ \dots \\ \text{R15} \end{bmatrix} + \begin{bmatrix} \begin{matrix} \{ \text{RAX} \\ \dots \\ \text{R15} \} \end{matrix} * \begin{bmatrix} 1 \\ 2 \\ 4 \\ 8 \end{bmatrix} \end{bmatrix} \right\} + [\text{displacement}]$$

[RIP]

- **RIP + Displacement** – зміщення (32-бітне) відносно RIP:

`mov EAX, [RIP + 0x1230]`

- ◇ Дуже важливий режим – потрібен для створення position independent коду, про який далі. Приклад:
<https://godbolt.org/z/K7TeYb96d>.
- Джерелом натхнення для прикладів вище служила, окрім офіційної документації Intel²⁷, стаття https://blog.yossarian.net/2020/06/13/How-x86_64-addresses-memory

²⁷Intel 64 and IA-32 Architectures Software Developer's Manual Combined Volumes: 1, 2A, 2B, 2C, 2D, 3A, 3B, 3C, 3D and 4, 4834 сторінки на листопад 2022 року.

Інші способи адресації x86-64

- Сегментні регістри FS та GS використовуються для доступу до TLS даних – що це таке, буде на АКС. Приклад: <https://godbolt.org/z/c6GErz79M>.
- Стрічкові інструкції використовують RSI як вказівник на вхідні дані, RDI – на вихідні, з автоматичним інкрементом використаних регістрів. Про ці команди – див. далі.
- Порти вводу-виводу мають свою адресацію – про неї теж пізніше.

Legacy адресація пам'яті – x86-32, x86-16

- В ISA x86-32 звертання до пам'яті може відбуватися так²⁸:

$$\left\{ \begin{array}{l} \text{CS :} \\ \text{DS :} \\ \text{SS :} \\ \text{ES :} \\ \text{FS :} \\ \text{GS :} \end{array} \right\} \left[\begin{array}{l} \text{EAX} \\ \text{EBX} \\ \text{ECX} \\ \text{EDX} \\ \text{ESP} \\ \text{EBP} \\ \text{ESI} \\ \text{EDI} \end{array} \right] + \left[\begin{array}{l} \text{EAX} \\ \text{EBX} \\ \text{ECX} \\ \text{EDX} \\ \text{EBP} \\ \text{ESI} \\ \text{EDI} \end{array} \right] * \left\{ \begin{array}{l} 1 \\ 2 \\ 4 \\ 8 \end{array} \right\} + [\text{displacement}]$$

- ◇ ESP не може бути індексним регістром, але у решті регістри рівноправні.

- x86-16:

$$\left\{ \begin{array}{l} \text{CS :} \\ \text{DS :} \\ \text{SS :} \\ \text{ES :} \end{array} \right\} \left[\begin{array}{l} \{ \text{BX} \} \\ \{ \text{BP} \} \end{array} \right] + \left[\begin{array}{l} \{ \text{SI} \} \\ \{ \text{DI} \} \end{array} \right] + [\text{displacement}]$$

- ◇ Видно спеціалізованість регістрів в "давні часи".
- ◇ **BX**, **BP** – базові регістри, **DI**, **SI** – індексні, **CX** – лічильник, **AX** – акумулятор, **DX** – вказівник для вводу-виводу, **SP** – вказівник стеку. В сучасних варіантах x86 ISA вони лише частково зберегли цю спеціалізацію – окремі команди працюють лише з конкретними регістрами.

²⁸Зображення взято з вікі: <http://en.wikipedia.org/wiki/X86.Displacement> – 8, 16 або 32 біти.

Розміри даних для x86

- Байт, db – спосіб оголосити в асемблері, GAS: .byte, .ascii, .asciiz, .zero <size> – size нульових байтів.
- 2 байти – слово, word, dw, GAS: .word.
- 4 байти – подвійне слово, dword, dd, GAS: .long, .float.
- 8 байт – четверне слово, qword, dq, GAS: .quad, .double.
- 10 байт – розмір (legacy) long double/extended, dt, GAS: .tfloat.
- 16 байт – параграф, (double quadword).
- $2^{10} = 1024$ – кілобайт.
- $2^{12} = 4096$ – 4кб, сторінка.
- $2^{16} = 65536$ – 64кб, (legacy) сегмент.
- $2^{20} = 1048567$ – 1Мб.
- і т.д. В більш чи менш екзотичних ситуаціях трапляються сторінки 2Мб, 4Мб, 1Gb.

Машинні команди – інструкції

- Користувачькі інструкції можна приблизно поділити на декілька типів.
- ◇ Операції над даними – завантаження з пам'яті, вивантаження в пам'ять, встановлення в регістри безпосередніх значень тощо.
- ◇ Арифметичні та логічні операції, включаючи операції порівняння.
- ◇ Керування потоком виконання – переходи, виклики функцій.
- ◇ Всілякі інші.
- Характерною особливістю x86 є інструкції, що одночасно звертаються до пам'яті та виконують арифметичну операцію, наприклад: "add [RSI], 5". Інші сучасні ISA, зазвичай, не мають таких команд²⁹.

²⁹Це одна із відмінностей між RISC і CISC різновидами ISA – детальніше будемо їх розглядати на АКС. ▶

Команди переміщення даних

- **mov** – перемістити дані. Може працювати із двома регістрами, регістром і пам'яттю або зберігати у регістр **безпосереднє значення** (immediate) – константу.
- Безпосереднє значення може бути 8, 16, 32 чи 64-бітне (в залежності від команди та режиму).
- Інші сучасні ISA часто накладають значно більш жорсткі обмеження на них, через бажання мати фіксовану довжину команди.
- Фізично, це багато різних команд. До прикладу:
 - ◇ Команди, що працюють із RAX/EAX/AX/AL мають коротші на байт варіанти кодування³⁰, і можуть бути швидшими.
 - ◇ Переміщення у/з системних регістрів – це не зовсім те, що переміщення між регістрами загального використання тощо.
- Оскільки наш огляд доволі поверхневий, не заглиблюватимемося у подробиці – якщо що, асемблер вас повідомить, що певна комбінація операндів – заборонена.
- Переміщення пам'ять-пам'ять відсутні (окрім стрічкових команд).
- В 64-бітному режимі запис в молодшу частину регістра зануляє старших 32 біти.

³⁰Хоча, враховуючи плутану систему команд x86, для них існують також "довгі" варіанти – на рівні асемблера команда та ж, а кодування на байт довше, як для якогось RBX.

- Приклади:

Копіювання безпосереднього значення (immediate),
іншого регістра чи пам'яті

```
mov    RAX, 15          # Reg<-imm
mov    RCX, RDX          # Reg<-reg
mov    RCX, [qword_var] # Reg<-mem
mov    [qword_var], RDX # Mem<-reg
```

Адреса може бути будь-якого із форматів адресації:

```
mov    RDX, [RBX + RSI*4]
mov    [qword_var], RCX
mov    RDI, [RBX + RSI*4 + 16]
```

Частини регістрів:

```
mov    CL,  DH
mov    AL,  [byte1]
mov    BX,  [word2]
mov    ECX, [dword3]
```

xchg – обмін даних

- xchg обмінює два регістри чи регістр і пам'ять місцями.
- Цікава властивість, що більше відношення має для курсу АКС – виконується атомарно³¹.

```
xchg    ECX, EDX  
xchg    BL, [byte2]  
xchg    [qword_var], RCX
```

³¹Інтел x86 взагалі в тому плані дуже "щедрий", будь-яку команду можна зробити атомарною за допомогою префіксу LOCK, але для xchg навіть він не потрібен

lea

- lea – load effective address. Спосіб помістити в регістр адресу, отриману будь-яким із способів адресації.
- Завантажує результат **обчислення адреси** в регістр! Не зважаючи на квадратні дужки, не звертається до пам'яті.
- Виглядає дивною – навіщо вона? Це перестає дивувати, якщо згадати, що обчислення адреси відбувається в окремому блоці процесора, не в ALU (Arithmetic Logic Unit), тому її використання може прискорювати обчислення.
- Трапляється достатньо часто – як хитра арифметична інструкція.

```
lea    EAX, [0x100]
```

```
lea    EDX, [EBX+2*EDI+0x100]
```

```
lea    EAX, [EAX+2*EAX+0x1] # EAX = 3*EAX + 1
```

- Reverse-engineering цього суматора в 8086: <http://www.righto.com/2020/08/reverse-engineering-adder-inside-intel.html>.

Інші

- **MOVZX** – перемістити із меншого аргументу (регістра чи пам'яті) у більший, заповнивши нульовими бітами старшу частину. Zero Extend, копіювання беззнакових чисел.
- **MOVSX** – Sign Extend, копіюючи в старші біти біт знаку. Дозволяє скопіювати знакове число з меншого операнда у більший.

```
mov      byte ptr [byte1], 0xF0      # 240 or -16
movzx    EAX, byte ptr [byte1] # EAX = 0x000000F0
movsx    EBX, byte ptr [byte1] # EBX = 0xFFFFFFFF
```

- Зверніть увагу на директиву вказання розміру даних перед адресою, `byte ptr [addr]` – без неї GAS не знатиме ваших намірів, принаймні у двох останніх випадках.
- ◇ Деякі асемблери пишуть "`byte ptr [addr]`".

Інші – рідко необхідні чи й шкідливі на сучасних процесорах

- `xlat` – заміняє байт в `AL` на байт із таблиці, адреса якої задана в `R|E|BX`, а індекс береться із `AL`. Корисна для всіляких однобайтових перекодувань.
- `lahf`, `sahf` – завантажити прапорці в регістр `AL` чи із нього – в регістр прапорців. Краще уникати.
- `lss`, `lds`, `les`, `lfs`, `lgs` – завантажує "довгу адресу", в сегментний регістр плюс регістр загального використання. Нерелевантні для сучасних систем, фактично відсутні в 64-бітовому режимі³², краще уникати.
- Те ж стосується команд переміщення із та в сегментні регістри: `"mov [thesegword], ES"`, `"mov ES, AX"`.
- Команди `BCD` корекції – `AAA`, `DAA` тощо.

³²Сегментні регістри `FS` і `GS` мають деяке використання і в сучасних ОС, тому підтримуються і в `x86-64`, решта – ні.

Керування прапорцями – користувацькі

- STC – Set carry flag.
- CLC – Clear the carry flag.
- CMC – Complement the carry flag.
- CLD – Clear the direction flag.
- STD – Set direction flag.
- Керування системними прапорцями не розглядаємо.

Стрічкові команди

- MOVS/MOVSb/MOVSsw/MOVSd/MOVSq – копіює дані, відповідного розміру (вказаного в суфіксі – B_yte, W_ord і т.д.), із [RSI] в [RDI].
- Якщо прапорець DF == 0, після цього обидва регістри збільшуються на 1/2/4/8, в залежності від формату команди. Якщо ж DF == 1, вони на ту ж величину зменшуються.
- Існують варіанти, що використовують пари ESI-EDI, SI-DI. Насправді, в таких випадках, копіюється із DS:[ESI] в ES:[EDI], але на сучасних системах сегменти нерелевантні.
- Їх зручно використовувати із **префіксом REP** – він змушує стрічкову команду повторюватися, зменшуючи CX/ECX/RCX на 1, поки він не стане рівним нулю.
- Використання цього префіксу для не стрічкових команд³³ – undefined behaviour.
- Копіюємо 16 8-байтових слів:

```
    cld          # Очищаємо прапорець напрямку
    lea          RSI, [vect_double_1]
    lea          RDI, [vect_double_2]
    mov          ECX, 0x10      # Старші біти в RCX занулилися
rep  movsq
```

³³Існують також стрічкові команди вводу-виводу – INS/OUTS.

Інші стрічкові інструкції

- Зауважте, що у всіх стрічкових інструкцій розмір операнду та розрядність адреси можна змінювати незалежно, хоча, не всі комбінації можливі, а їх вибір залежить від режиму.
- Сімейство **CMPS** (із всіма можливими суфіксами) порівнює стрічки, відповідним чином виставляючи прапорці.
- **SCAS** – порівнює вміст AL/AX/EAX/RAX із пам'яттю за адресою [DI]/[EDI]/[RDI].
- **LODS** – завантажує 1/2/4/8 байт із [SI]/[ESI]/[RSI] в AL/AX/EAX/RAX.
- **STOS** – зберігає AL/AX/EAX/RAX в пам'ять, за адресою [DI]/[EDI]/[RDI].
- Безумовний префікс не дуже корисний для цих команд, але існує два із умовою (для кожного – по дві мнемоніки).
- REPE/REPZ префікси, крім занулення RCX/ECX, зупиняються, коли ZF=0.
- REPNE/REPNZ префікси, крім занулення RCX/ECX, зупиняються, коли ZF=1.
- Ще підказка для пам'яті: SI – Source Index, DI – Destination Index.

Арифметичні інструкції

- Арифметичні команди виконують обчислення над регістрами загального використання, із залученням безпосередніх значень та пам'яті.
- Всі арифметичні команди змінюють прапорці згідно результату (CF – якщо стався перенос, ZF – якщо результат нульовий тощо). Виняток – INC та DEC не змінюють CF, тому на сучасних процесорах не в фаворі.
- **Ремарка:** Інші сучасні архітектури намагаються, щоб команди так бездумно прапорці не змінювали! Детальніше, чому це важливо — поговоримо на курсі АКС.

Додавання

- **add** – обчислює суму своїх аргументів і поміщає результат в перший.
- **adc** – обчислює суму своїх аргументів і додає до неї значення прапорця переносу CF.
- **inc** – додає до свого аргументу одиницю. Навіть у випадку переповнення не встановлює CF,

```
add    eax, 17      # Додаємо безпосереднє значення
add    ebx, eax      # Додаємо вміст регістра
add    ebx, [esi]    # Додаємо вміст пам'яті
add    [edi], ebx    # Додаємо до вмісту пам'яті
```

```
mov    ebx, 0x10
clc                                # Очищаємо CF:      CF=0
adc    ebx, 0x5      # ebx := ebx+5+CF == 15h
```

```
mov    ebx, 0x10
stc                                # Встановлюємо CF: CF=1
adc    ebx, 0x5      # ebx := ebx+5+CF == 16h
```

```
inc    esi                # esi:=esi+1;
```

Приклад: додавання двох 64-бітових чисел в 32-бітовому режимі

```
# Завантажуємо в eax, ebx молодші частини 64-бітових чисел
mov     eax, dword ptr [largearg]
mov     ebx, dword ptr [large2arg]
# Завантажуємо в ecx, edx старші частини 64-бітових чисел
mov     ecx, dword ptr [largearg+4]
mov     edx, dword ptr [large2arg+4]

add     eax, ebx
adc     ecx, edx
# Тепер в парі eax:ecx -- результат в( eax -- молодша частина)

mov     dword ptr [large_sum] , eax
mov     dword ptr [large_sum+4], ecx
```

.....

```
largearg:    .quad  0x346586346346
large2arg:   .quad  0x346586346345 # Відрізняється на 1
large_dif:   .quad  0
large_sum:   .quad  0
```

Віднімання

- **sub** – Віднімає другий аргумент від першого і поміщає результат в перший.
- **sbb** – Віднімає другий аргумент від першого, враховуючи можливість позичання, на наявність чи відсутність якого вказує CF.
- **dec** – Віднімає від свого аргументу одиницю. Не встановлює CF.
- **cmp** – Команда порівняння. Фактично виконує віднімання, виставляє відповідним чином прапорці але відкидає результат!

```
sub    eax, 17      # Віднімаємо безпосереднє значення
sub    ebx, eax      # Віднімаємо вміст регістра
sub    ebx, [esi]    # Віднімаємо вміст пам'яті
sub    [edi], ebx    # Віднімаємо від вмісту пам'яті
```

```
mov    ebx, 0x9
cfc    # Очищаємо CF:      CF=0
sbb    ebx, 0x5      # ebx := ebx-5-CF == 4h
```

```
mov    ebx, 0x9
stc    # Встановлюємо CF: CF=1
sbb    ebx, 0x5      # ebx := ebx-5-CF == 3h
```

- ```
dec esi # esi:=esi-1;

clc # CF=0
xor eax, eax # Короткий еквівалент mov eax,0h
dec eax # eax:=0FFFFFFFFh, відбулося позичання, АЛЕ CF=0

clc # CF=0
mov eax, 0;
sub eax, 1 # eax:=0FFFFFFFFh, відбулося позичання, CF=1
```

## Приклад: віднімання двох 64-бітових чисел в 32-бітовому режимі

```
Завантажуємо в eax, ebx молодші частини 64-бітових чисел
mov eax, dword ptr [largearg] # dword тут важливий
mov ebx, dword ptr [large2arg]

Завантажуємо в ecx, edx старші частини 64-бітових чисел
mov ecx, dword ptr [largearg+4]
mov edx, dword ptr [large2arg+4]

sub eax, ebx
sbb ecx, edx
Тепер в парі eax:ecx -- результат в(eax -- молодша частина)

mov dword ptr [large_dif] , eax
mov dword ptr [large_dif+4], ecx
```

.....

```
largearg: .quad 0x346586346346
large2arg: .quad 0x346586346345
large_dif: .quad 0
large_sum: .quad 0
```

## Про прапорці переносу та переповнення

- CF – вказує, чи відбувся перенос за межі розміру операнду (регістра чи пам'яті).
- OF – служить для діагностування знакового переповнення. Однак, з ним все не так просто.

- ```
mov al, 0b00111111
mov bl, 0b00000001
```

```
add al, bl
```

```
# результат - 0100 0000b
```

```
# Тому CF=0, OF=0
```

```
mov al, 0b01000000
```

```
mov bl, 0b01000000
```

```
add al, bl
```

```
# Результат 1000 0000b
```

```
# Старший біт дорівнює 1.
```

```
# Переносу за розрядну сітку не було
```

```
# CF=0, OF=1
```

```
# Вважаємо беззнаковими: 64 + 64 = 128 -- правильно
```

```
# Вважаємо числами зі знаком (+64) + (+64) = -128, неправильно.
```

- Перенос без переповнення —перенос одиниці і в знаковий розряд і за межі розрядної сітки.
- Результат, **увага**: CF=1, OF=0

```
mov al, 0b11111100
mov bl, 0b00000101
add al, bl
# Результат: 0000 0001b
# 252 + 5 = 1 --- неправильно
# -4 + 5 = 1 --- правильно
# CF=1, OF=0
```

- Ще варіант:

```
mov al, 0b11100000
mov bl, 0b11100000
add al, bl
# Результат -- 1100 0000b
# 224 + 224 = 192 --- неправильно
# -32 + (-32) = -64 --- правильно
# CF=1, OF=0
```

- Перенос з переповненням — перенесення тільки за межі розрядної сітки. Результат: CF=1, OF=1.

```
mov al, 0b10000000
```

```
mov bl, 0b10000000
```

```
add al, bl
```

```
# Результат 0000 0000b
```

```
# 128 + 128 = 0 --- неправильно
```

```
# -128 - 128 = 0 --- неправильно
```

- Описане вище можна зобразити у вигляді наступної таблиці:

Перенесення за розрядну сітку	перенесення в старший (знаковий) біт	Прапорці	Беззнакове правильне	Знакове правильне
або				
Позичання у старший біт	позичання з старшого біта			
Hi	Hi	CF=0, OF=0	Так	Так
Hi	Так	CF=0, OF=1	Так	Hi
Так	Hi	CF=1, OF=1	Hi	Hi
Так	Так	CF=1, OF=0	Hi	Так

- CF вказує на беззнакове переповнення – і невірний результат. OF – те ж, для знакових чисел.
- CF встановлюється якщо є перенесення за розрядну сітку, а OF отримується в результаті логічної операції XOR над першим і другим стовпцем — наявністю перенесення за розрядну сітку і перенесення в старший біт³⁴.
- Приклади базуються на статті Чугайнової Н.Г., "Перенос и переполнение - что они представляют собой на самом деле?" (див., наприклад, http://web.archive.org/web/20190428222546/http://www.allasm.ru/proc_08.php).

³⁴Або XOR значення CF і наявності перенесення в старший біт.

Множення і ділення — mul, imul, div, idiv

- mul та div виконують операції над беззнаковими величинами. Тобто найстарший біт не вважається знаковим.
- imul і idiv – їх аналоги для знакових чисел.
- mul та imul множать число із EAX на операнд, зберігають результат в EDX:EAX. Те ж для RAX->RDX:RAX, AX->DX:AX, AL->AX.
- div та idiv ділять число, збережене в EDX:EAX на переданий операнд, зберігають частку в EAX, остачу в EDX. Те ж для RDX:RAX; DX:AX та AX (AH:AL).
- ◇ Спричиняють виключення "ділення на нуль", якщо ділиться на нуль, або результат не поміщається в AL/AX/EAX/RAX.

```

mov     EDX, 0x02
mov     EAX,    0
mov     EBX, 0x40
idiv    EBX      # 2 00000000h/40h = 80000000h
              # EAX=80000000h, EDX=0h

mov     EDX, 0x02
mov     EAX,    0
mov     EBX, 0x51
div     EBX      # 2 00000000h/51h = 06522C3Fh
              # EAX=06522C3Fh, EDX=11h
  
```

Інші арифметичні та логічні команди

- neg – змінити знак на протилежний.
- not, and, or, test³⁵, xor – відповідні побітові операції.
- shl, shr – "прості" бітові зсуви.
- rol, ror – "прості" бітові ротації.
- rcl, rcr – ротації через CF.
- sal, sar – арифметичні зсуви, не псуватимуть від'ємні числа (теж через CF).
- shld, shrd – подвійні зсуви (біти для заповнення нових позицій беруться із операнда).
- bt, bts, btr, btc, bsf, bsr – маніпуляції окремими бітами.
- cbw, cwd, cwde, cdq – різноманітні знакові розширення, не рекомендуються.

³⁵Виконує and, встановлює відповідним чином прапорці, але не зберігає результат.

Команди переходу

- На x86 їх є дуже багато – "довгі" (far), "близькі" (near), "короткі" (short), і це – не рахуючи 64-бітного режиму.
- заради лаконічності, не будемо зупинятися на таких подробицях – зазвичай, асемблер здатен обрати оптимальний варіант.
- Безумовний перехід `jmp`:

```
                                jmp shortOrNearLabel  
labelAfterCommand:  
                                <some commands>  
  
shortOrNearLabel:  
                                <some other commands>  
  
                                mov  ebx,NearLabel  
                                jmp  [ebx]  
labelAfterCommand:  
                                <some commands>  
  
NearLabel:  
                                <some other commands>
```

Відступ

- Головний перл практичних в 2021 році був, звичайно:

```
    jmp label  
label:
```

От навіщо?! Цей стрибок туди, куди виконання і так зараз перейде, але місце в коді і час процесора займає.

Умовні переходи

- Команди умовних переходів здійснюють "стрибок", якщо встановлено відповідні прапорці.
- Для зручності поміж них є синоніми, названі згідно логіки переходів, а не конкретних прапорців.
- Є також умовні переходи по вмісту CX/ECX/RCX.
- Приклад:

```
    cmp     eax, 0
    je      areEqual      # Те ж що i jz
    # <some commands>    # Тут якщо вміст eax не дорівнює нулю
areEqual:
    # <some other commands>
    cmp     eax, 0
    jne     areNotEqual   # Якщо не дорівнює нулю
    jmp     someLabel
areNotEqual:
```

- jz – Jump-if-Zero(flag) та je – Jump-if-Equal, це дві назви для однієї і тієї ж команди.
- ◇ Назва jz каже, що вона перевіряє прапорець ZF.
- ◇ "Cmp" віднімає свої операнди, відкидаючи результат, але виставляючи прапорці, тому, коли аргументи рівні, встановлює ZF в 1. Звідти ідея: "стрибнути, якщо рівні".

je	Jump if Equal — перехід, якщо прапорець нуля (Z) рівний 1. Синонім jz , протилежна jne .	ZF=1
jz	Jump if Zero — виконує перехід, якщо прапорець нуля (Z) рівний 1. Синонім je , протилежна jnz .	ZF=1
jcxz	Jump if CX Zero	CX=0
jecxz	Jump if ECX Zero	ECX=0
jp	Jump if Parity – перейти якщо PF=1. Синонім jpe , протилежна — jnp .	PF=1
jpe	Jump if Parity Even – перейти якщо результат містить парну кількість біт. Синонім jp , протилежна — jpo .	PF=1.
jne	Jump if Not Equal. Синонім jnz , протилежна je .	ZF=0
jnz	Jump if Not Zero. Синонім jne , протилежна jz .	ZF=0
jnp	Jump if Not Parity – перейти якщо PF=0. Синонім jpo , протилежна — jp .	PF=0
jpo	Jump if Parity Odd – перейти якщо результат містить непарну кількість біт. Синонім jnp , протилежна — jpe .	PF=0

Беззнакові операції

ja	Jump if Above. Після стр, якщо перший аргумент більший від другого (вважаючи їх беззнаковими), то прапорці набувають якраз таких значень. Синонім jnb , протилежна jna .	CF=0, ZF=0
jae	Jump if Above or Equal. Синонім jnb , jnc , протилежна jnae .	CF=0
jb	Jump if Below. Синонім jnae , jc , протилежна jnb .	CF=1
jbe	Jump if Below or Equal. Синонім jna , протилежна ja .	CF=1 або ZF=1
jc	Jump if Carry. Синонім jae , jnb , протилежна jnc .	CF=1
jnc	Jump if Not Carry. Синонім jb , jnae , протилежна jc .	CF=0
jna	Jump if Not Above. Синонім jbe , протилежна ja .	CF=1 або ZF=1
jnae	Jump if Not (Above or Equal). Синонім jb , jc , протилежна jae .	CF=1
jnb	Jump if Not Below. Синонім jae , jnc , протилежна jb .	CF=0

Беззнакові операції

jnbe	Jump if Not (Below or Equal). Синонім ja , проти- лежна jbe .	CF=0, ZF=1
------	--	---------------

Знакові операції

jg	Jump if Greater. Синонім jnle . Протилежна jng .	ZF=0, SF=OF
jge	Jump if Not (Greater or Equal). Синонім jnl . Протилежна jnge .	SF=OF
jl	Jump if Less. Синонім jnge . Протилежна jnl .	SF≠OF
jle	Jump if Less or Equal. Синонім jng . Протилежна: jnle .	ZF=1 або SF≠OF
jng	Jump if not Greater. Синонім jle . Протилежна jl	ZF=1 або SF≠OF
jnge	Jump if not Greater or Equal. Синонім jl . Протилежна: jge	SF≠OF
jnl	Jump if Not Less. Синонім jge . Протилежна jl .	SF=OF
jnle	Jump if Not Less or Equal. Синонім jl . Протилежна: jle .	ZF=0, SF=OF
jo	Jump if Overflow. Протилежна: jno	OF=1
jno	Jump if Not Overflow. Протилежна: jo	OF=0

Знакові операції

js	Jump if Sign — перейти якщо встановлено знаковий біт, відповідає від'ємному результату. Протилежна: jns	SF=1
jns	Jump if Not Sign — відповідає додатному числу. Протилежна: js	SF=0

- Див. також: <http://unixwiz.net/techtips/x86-jumps.html> – "Intel x86 JUMP quick reference"

Цикли

- Сімейство команд: `loop`, `loope`, `loopz`, `loope`, `loopnz`.
- При виконанні зменшує `CX/ECX/RCX` на 1, якщо він не 0 – здійснює стрибок.
- Крім `loop`, решта перевіряють ще `ZF`.
- Приклад, цикл на три ітерації:

```
    mov    ECX, 3
myloop:
    команди< циклу>

    loop   myloop
```

- Сучасні компілятори уникають використовувати цю команду.

Способи обійтися без переходів

- На сучасних процесорах³⁶ переходи достатньо дорогі.
- Тому існують команди обійтися без них.
- Це `cmovXX` – умовні переміщення та `setXX` – умовні встановлення байта (в `0x00` або `0x01`), де `XX` – прапорці умови, розглянуті вище.

```
cmovz    ECX, EDX # Якщо ZF = 1, скопіювати EDX в ECX.  
setc     AL       # Встановити AL=1, якщо CF=1; інакше AL=0.  
setnz    [byte2]
```

- В інших архітектурах (ISA) також існує цілий клас команд, які виконуються за певної комбінації прапорців та вважаються пор – порожньою командою, в інших випадках.

³⁶Зокрема, через спекулятивне виконання і неідеальність передбачення переходів.

Команда `por`

- Найпростіша команда — `por`.
- Дуже цінна команда, вона не робить нічого. `por` — No Operation.
- Займає один байт в пам'яті і має код `90h`. Не впливає ні на які прапорці.
- Фактично, це синонімом команди `xchg (R/E)AX, (R/E)AX`, але сучасні процесори оптимізовані, щоб ігнорувати її.
- Використовується для заповнення порожнього місця (вирівнювання, резервація місця) або затирання якихось інших команд (відладка).
- ◇ Ремарка: оскільки на `x86-64` запис у нижніх 32 біти 64-бітного регістра зануляють верхніх 32, на `x86-64` "`mov EAX, EAX`" не є `por`, оскільки занулить верхню половину `EAX` (а ось "`mov RAX, RAX`" є).

Розглянуті вище команди — навіть не початок! Детальніше див. тут:
<http://ref.x86asm.net/>, скажімо у варіанті `coder64`.

Ідіоми асемблера

- Типова ідіома – замість:

```
mov rax, 0
```

писати:

```
xor rax, rax
```

Опкоди:

```
0: 48 c7 c0 00 00 00 00    mov    rax,0x0
```

```
7: 48 31 c0                xor     rax,rax
```

- Вона настільки типова, що сучасні процесори про неї знають і оптимізують (будемо вивчати на АКС).

- Аналогічно, зазвичай, замість:

```
cmp rax, 0
```

варто використовувати:

```
test rax, rax
```

Опкоди:

```
0: 48 83 f8 00            cmp     rax,0x0
```

```
4: 48 85 c0                test    rax,rax
```

Приклад коду x87 FPU

- Зверніть увагу на поведінку стеку:

```
fld      qword [vdouble1]    # Load double
# ST0 = vdouble1
fld      qword [vdouble2]    # Load double
# ST0 = vdouble2, ST1 = vdouble1
faddp    # ST0 <= ST0 + ST1, ST1 - empty
fst      qword [vdouble3]
```

.....

```
vdouble1      dq  1.23
vdouble2      dq  4.56
vdouble3      dq  0.0
```

- З GAS будьте обережними із відніманням:

```
fsubp    # ST0 <= ST1 - ST0, ST1 - empty
```

- Приклад вище працює коректно для синтаксису Intel, однак в AT&T варіанті аргументи буде переставлено:
<http://www.mindfruit.co.uk/2012/03/trouble-with-fsub.html>,
https://sourceware.org/binutils/docs/as/i386_002dBugs.html

Scalar SSE2 floating point

- Використовуємо лише молодші частини регістрів:

```
MOVSD   XMM0, [vdouble1]
MOVSD   XMM1, [vdouble2]
ADDSD   XMM0, XMM1
MOVLPD  [vdouble3], XMM0
```

.....

```
vdouble1: .double 1.23
vdouble2: .double 4.56
vdouble3: .double 0.0
```

- В **MOVSD** та **ADDSD**, S – scalar, D – double.

Vector SSE2 floating point

- Приклад додавання двох double однією командою. (Детальніше – на АКС).

```
MOVAPD XMM0, [vect_double_1]
MOVAPD XMM1, [vect_double_2]
ADDPD  XMM0, XMM1
MOVAPD [vect_double_3], XMM0
```

```
.....
```

```
.align 16
```

```
vect_double_1 .double 1.23, 3.45
```

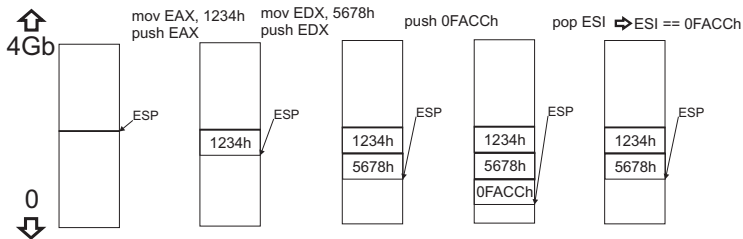
```
vect_double_2 .double 1.11, 2.22
```

```
vect_double_3 .double 0.0 , 0.0
```

- Дані для MOVAPD мають бути вирівняні на 16 байт – для цього використано директиву align 16.
- **MOVAPD** – **MOV** **A**ligned **P**acked **D**oubles.
- ◇ Існують також команди для неvirівняних звертань, типу MOVUPD, але вони можуть бути суттєво повільнішими.

Організація стеку – x86-32

- Поточна вершина стеку знаходиться в SP/ESP/RSP. На далі, вважатимемо, для простоти, що режим 32-бітовий³⁷.
 - Коли виконується команда `push`, від ESP віднімається 4 та за адресою [ESP] зберігається значення, яке `push`-ається в стек.
 - Тобто, стек росте **вниз** – від більших адрес до менших.
 - `pop` поводитьсь зворотним чином – в свій операнд кладе [ESP] і збільшує ESP на 4.
- ◇ `push ESP` зберігає в стеку значення ESP до операції.



- Для x86-64 – замість ESP всюди буде RSP, а замість 4, буде додаватися та відніматися 8.

³⁷Адреси в прикладах будуть коротшими, ніж в 64-бітовому, а суть та ж.

Керування стеком

- **push** — покласти значення або вміст регістру чи пам'яті в стек.
- **pop** — забрати значення з вершини стеку в регістр або пам'ять.
- **pushf** – зберегти регістр прапорців в стек.
- **popf** – відновити регістр прапорців зі стеку.
- **pusha/pora** – зберегти чи добути із стеку цілий список регістрів.
Не рекомендується до використання.
- Розрядність значення, що зберігається³⁸ визначається розрядністю коду³⁹.

push	EAX
push	[EDI]
push	0x333

pop	[ESI]
pop	EBX

³⁸Навіть, скажімо, для сегментних регістрів.

³⁹Можна примусово зробити 16-бітовою в 32 та 64-бітових режимах, але це екзотика. 

Виклик підпрограми

- **call** — виклик підпрограми:

```
call    0x12345678    # Виклик за безпосередньою адресою
call    [EBX]         # Опосередкований виклик,
                      # адреса підпрограми в EBX
```

Під час виклику адреса повернення кладеться на стек.

- **ret** — повернутися із підпрограми. Адреса повернення береться із стеку.

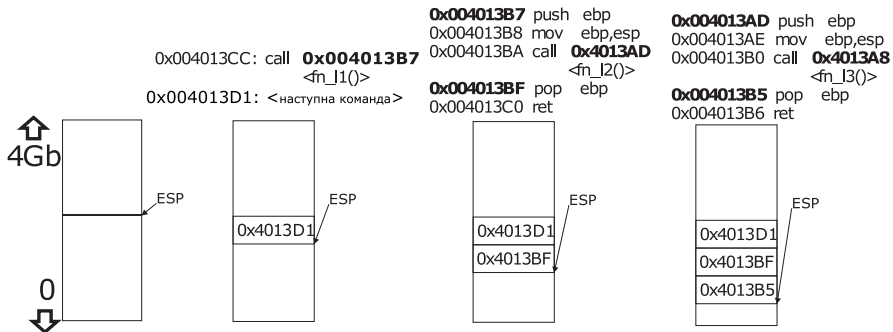
```
ret      # Взяти зі стеку останню збережену адресу
ret      4  # Після того, як зі стеку дістали адресу
           # повернення, відкинути ще вказану кількість
           # байт
```

Виклик функції

Розглянемо наступний код:

```
void fn_l3(){  
}  
  
void fn_l2(){  
    fn_l3();  
}  
  
void fn_l1(){  
    fn_l2();  
}  
  
int main(){  
    fn_l1();  
  
    return 0;  
}
```

Виклик функції (32-бітовий код)



Calling conventions — угоди про виклики

- Угоди про виклики — домовленість про те, як передаватимуться у функцію параметри, та як повертатиметься результат, як кодуються імена функцій в об'єктних файлах, тощо.
- Їх існує багато, стандартизовані вони лише в загальних рисах.
- ◇ При тому, код, який викликає функцію та сама функція повинні дотримуватися єдиних домовленостей.
- ◇ Зазвичай про це турбується компілятор, однак слід бути уважним, особливо, якщо користуєтеся скомпільованими не вами модулями.
- Декілька прикладів угод про виклики: cdecl, syscall, pascal, stdcall, fastcall (є варіанти від Microsoft та Borland), thiscall, і т.д. і т.п.
- ◇ Коротко можна почитати тут:
http://en.wikipedia.org/wiki/Calling_convention,
http://en.wikipedia.org/wiki/X86_calling_conventions,
- ◇ а детальніше тут: <http://www.agner.org/optimize/> — "Calling conventions for different C++ compilers and operating systems" by Agner Fog — детальний, повний та доволі точний довідник.
- Розпочнемо розгляд з cdecl, типової угоди про виклики для 32-бітного коду (x86) на C.

cdecl for x86-32

- Аргументи функціям передаються через стек.
- Аргументи передаються справа наліво.
- Цілі числа та вказівники повертаються за допомогою регістра EAX.
- ◇ Числа з рухомою комою повертаються через спеціальний регістр співпроцесора, ST0.
- Функція може вільно змінювати EAX, ECX і EDX, (так звані scratch-регістри, або caller-saved) але зобов'язана зберегти і відновити всі інші регістри, якщо модифікує їх (callee-saved)⁴⁰.
- За очищення стеку від параметрів відповідає код, що викликає функцію (caller)
- Про передачу більш складних типів коротко буде сказано пізніше, але важливо знати, що власне у її правилах різні компілятори поведуться трішки по різному, навіть в рамках cdecl.
- В деяких компіляторах можна змусити використовувати cdecl ось так:

```
void _cdecl fn();
```

- ◇ `_cdecl` — не є частиною стандарту C/C++! Це платформно-залежне розширення, хоча й популярне.

⁴⁰Регістри співпроцесора мають бути порожніми на вході, зайнятим на виході може бути хіба ST0, якщо функція повертає число з рухомою комою.

Розглянемо функцію:

```
void fn(int a, int b, int c);
```

Викличемо її, для простоти, так:

```
fn(1, 2, 3);
```

Передача аргументів їй може відбуватися якимось так:

Безпосередньо захвати
в стек:

```
push    3
push    2
push    1
call    <fn>
```

Трішки зекономити:

```
sub     esp, 12    ; 4*3
mov     [esp+8], 3
mov     [esp+4], 2
mov     [esp],   1
call    <fn>
```

GCC вимагає, щоб стек
був вирівняний на ме-
жу 16 байт, тому виді-
ляє зразу $16 = 10h$

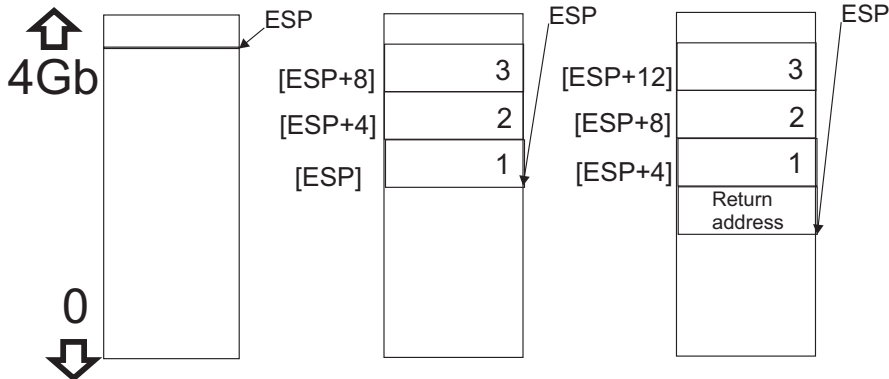
```
sub     esp, 0x10 ; 16
mov     [esp+8], 3
mov     [esp+4], 2
mov     [esp],   1
call    <fn>
```

Ремарка: команду "mov [esp+8], 3" в коді потрібно записувати якимось
так: "mov word ptr [esp+8], 3"

Яким би способом аргументи не були покладені в стек, картина виглядатиме так:

Кладемо
аргументи
в стек

Викликаємо
функцію



Таким чином, відштовхуючись від значення ESP на початку її виконання, функція завжди може знайти свої аргументи.

Доступ до аргументів

- Подивимося на це з точки зору функції:

```
void fn(int a, int b, int c)
{
    a = a + b + c;
}
```

- ◆ Для спрощення обійдемося поки без локальних змінних.
- Компілятор може згенерувати код ось такого типу⁴¹ (<https://godbolt.org/z/88baYY3o3>):

```
push    ebp
mov     ebp, esp
```

```
mov     eax,DWORD PTR [ebp+0x0C]
mov     edx,DWORD PTR [ebp+0x08]
add     eax,edx
add     eax,DWORD PTR [ebp+0x10]
mov     DWORD PTR [ebp+0x08],eax
```

```
pop    ebp
ret
```

⁴¹PTR – це такі "красивості", які люблять деякі асемблери, зокрема – GNU GAS. ▶

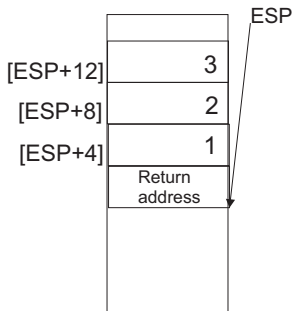
Кадр стеку — stack frame

Розглянемо наступний код:

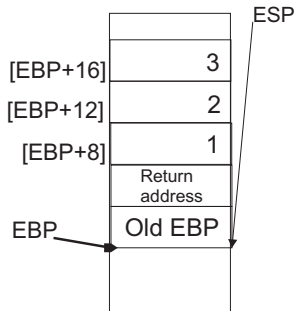
```
push    ebp        # Зберігаємо EBP
mov     ebp, esp    # Налаштовуємо кадр стеку
.....
pop     ebp        # Відновлюємо EBP
ret
```

- ESP буде змінюватися в процесі роботи функції, тому хоча локальні параметри на початку функції починаються з ESP+4, потім доступатися до них може бути незручно.
- Тому варто десь зберегти його значення на початку функції.
- Архітектура x86 рекомендує використовувати для цього регістр EBP.
- Однак, згідно угоди про виклики cdecl, його зберігати та відновлювати повинна сама функція. Тому, вона починає із того, що зберігає EBP в стеку, а завершує, перед виходом, відновлюючи його.
- Кадри стеку дуже важливі для зневадження, розкрутки виключень, профайлінгу. Див., наприклад: "[The Return of the Frame Pointers](#)", 2024.

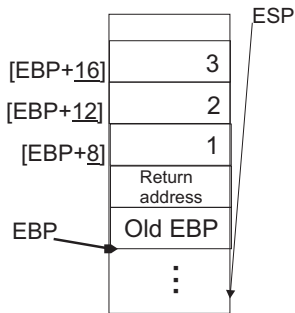
На початку
функції



push EBP
mov EBP, ESP



Тіло функції
.....



Зауважте, звертатися до аргументів функції можна не залежно від змін ESP.

Нагадаємо собі реалізацію функції, яку розглядаємо:

```
void fn(int a, int b, int c)
{
    a = a + b + c;
}
```

```
push    ebp
mov     ebp,esp

mov     eax,DWORD PTR [ebp+0x0C] # 0x0C = 12, b
mov     edx,DWORD PTR [ebp+0x08] # 0x08 = 8,  a
add     eax,edx
add     eax,DWORD PTR [ebp+0x10] # 0x10h = 16, c
mov     DWORD PTR [ebp+0x08],eax # Зберегти в a

pop     ebp
ret
```

Завершуючи із передачею простих аргументів

```
fn_3args(1,2,3);
```

; Готуємо аргументи функції

```
mov    dword ptr [esp+0x8],0x3 ; Третій
mov    dword ptr [esp+0x4],0x2 ; Другий
mov    dword ptr [esp],0x1     ; Перший
call   0x4013ce <fn_3args(int, int, int)>
```

Зрозуміло, що, із ввімкненою оптимізацією, компілятор викине зайві переміщення туди-сюди. Але без оптимізації найбільш наочно видно його наміри та принципи роботи.

```
int a1 = 5, b1 = 55, c1 = 555;
fn_3args(a1,b1,c1);
```

; Ініціалізуємо змінні

```
mov    dword ptr [esp+0x18],0x5
; esp+0x18 - адреса a1
mov    dword ptr [esp+0x14],0x37
; esp+0x14 - адреса b1
mov    dword ptr [esp+0x10],0x22b
; esp+0x10 - адреса c1
```

; Готуємо аргументи функції

```
mov    eax,dword ptr [esp+0x10]
mov    dword ptr [esp+0x8],eax ; Третій
mov    eax,dword ptr [esp+0x14]
mov    dword ptr [esp+0x4],eax ; Другий
mov    eax,dword ptr [esp+0x18]
mov    dword ptr [esp],eax     ; Перший
; Ремарка: місце під аргументи
; компілятор зарезервував раніше,
; до початку цього коду
call   0x4013ce <fn_3args(int, int, int)>
```

Повернення значень простих типів

```
int fn_3args_ret
    (int a, int b, int c){
    a = a + b + c;
    return a;
}
```

```
push    ebp
mov     ebp,esp
mov     eax,dword ptr [ebp+0xc]
mov     edx,dword ptr [ebp+0x8]
add     eax,edx
add     eax,dword ptr [ebp+0x10]
mov     dword ptr [ebp+0x8],eax
mov     eax,dword ptr [ebp+0x8]
pop     ebp
ret
```

```
int d;
d = fn_3args_ret(1,2,3);
```

```
mov     dword ptr [esp+0x8],0x3
mov     dword ptr [esp+0x4],0x2
mov     dword ptr [esp],0x1
call    0x4013e1
        <fn_3args_ret(int, int, int)>
mov     dword ptr [esp+0x1c],eax
```

Локальні змінні

Код вище вже містить натяки, що локальні змінні теж знаходяться в стеку.

Розглянемо модифікацію нашої функції:

```
int fn_locals(int a, int b, int c)
{
    int r;
    r = a + b + c;
    return r;
}
```

```
push    ebp
mov     ebp, esp

sub     esp, 0x10

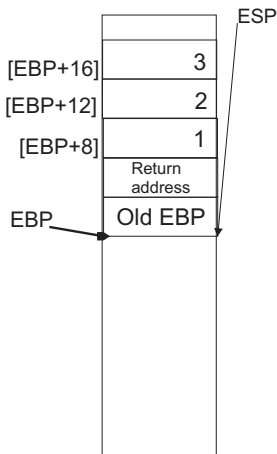
mov     eax, dword ptr [ebp+0xc]
mov     edx, dword ptr [ebp+0x8]
add     eax, edx
add     eax, dword ptr [ebp+0x10]

mov     dword ptr [ebp-0x4], eax

mov     eax, dword ptr [ebp-0x4]

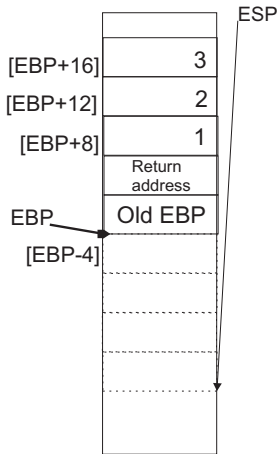
mov     esp, ebp
pop     ebp
ret
```

```
push EBP  
mov EBP, ESP
```

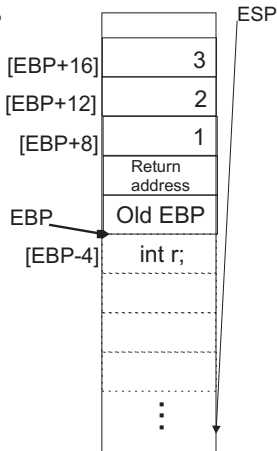


Виділяємо місце
під локальні змінні:

```
sub ESP, 0x10
```



Звертання до
локальної змінної
`mov eax, [EBP-0x4]`



Віднявши від ESP – резервуємо місце в стеку, додавши – звільняємо,
відкидаючи вміст.

Готуємо кадр стеку:

Виділяємо пам'ять для локальних змінних⁴²:

Зберігаємо результат в локальній змінній `r`, до якої звертаємося відносно границі кадру стеку:

Кладемо в `EAX` результат роботи із локальної змінної:

Очищаємо стек від локальних змінних та відновлюємо `EBP`. Синонім – `leave`.

Завершуємо функцію

```
push    ebp
mov     ebp, esp

sub     esp, 0x10

; Розрахунок результату
mov     dword ptr [ebp-0x4], eax

mov     eax, dword ptr [ebp-0x4]

mov     esp, ebp
pop     ebp
leave

ret
```

⁴²x86 має спеціальну команду для такого, `enter`. Ці три команди будуть еквівалентні до `enter 0x10`, однак, остання відверто повільна на сучасних процесорах, тому не використовується. Згадана нижче `leave` – достатньо ефективна.

Коротко про передачу структур та класів

- Детально тему не розглядатимемо, якщо цікаво — див. згадану роботу "Calling conventions for different C++ compilers and operating systems", розділ "7.1 Passing and returning objects". Нижче лише кілька прикладів для ілюстрації.
- 32-бітовий VS передає всі структури та класи, просто копіюючи їх в стек.
- 32-бітовий GCC копіює структури та класи в стек повністю, крім випадків коли клас має конструктор копіювання, деструктор або віртуальні функції — тоді він створює копію дець в пам'яті та кладе в стек вказівник на цю копію⁴³.
- Не статичні методи класів отримують вказівник на об'єкт як "нульовий", неявний, аргумент.
- Конструктор повертає вказівник на об'єкт в регістрі, який відповідна calling convention використовує для повернення аргументів.
- Якщо об'єкт неможливо повернути регістрами, вони зберігаються у місці, виділеному caller-функцією, вказівник на яке передається як неявний аргумент. Подробиці відрізняються між платформами.

⁴³Нагадаю, що для програміста все це відбувається повністю прозоро.

Щодо "виконавчого" стеку

- Через загрозу buffer overflow, стек, доступний для виконання – небезпечний.
- Тому, починаючи з binutils версії 2.39 (2022), лінкер скаржиться на ситуацію, де мусить створити такий стек:

```
ld: warning: hello_linux32_libc_pic.o: missing .note.GNU-stack section implies \
executable stack
ld: NOTE: This behaviour is deprecated and will be removed in a future version \
of the linker
```
- Для асемблерної програми виправляється якимось таким рядком:

```
.section .note.GNU-stack,"",@progbits
```

Він забороняє "виконавчий" стек.
- Якщо ж такий стек необхідний, слід лінкеру передати "-z execstack".
- ◇ Він потрібен, наприклад, для вкладених функцій – нестандартного розширення GCC.
- "Зворотні" опції:

```
.section .note.GNU-stack,"x",@progbits
```

та "-z noexecstack".
- Деякі подробиці див. [цю відповідь на Stackoverflow](#) та ["The linker's warnings about executable stacks and segments"](#).

Return Oriented Programming (ROP)

- Навіть коли стек заборонений до виконання, за допомогою переповнення стеку все ще можна виконати зловмисний код.
- ◇ Така техніка – Return Oriented Programming (ROP), підстановка таких адрес повернення, щоб фрагменти коду, який вже є в адресному просторі процесу ("гаджети"), виконали потрібне зловмиснику.
- ASLR (Address space layout randomization), зокрема, є способом боротися з ROB.
- Сучасні процесори намагаються вибудувати засоби захисту від таких зловмисників: валідація (підписування) адрес, тіньовий стек (shadow stack), як програмний, так і апаратний.
- Суміжні підходи: дозволяти системні виклики лише з певних модулів.
- Серія статей про ці технології:
 - ◇ "A quick introduction to return address protection technologies"
 - ◇ "Effects of classic return address tricks on hardware-assisted return address protection"
 - ◇ "Evaluating tail call elimination in the face of return address protection, part 1"
 - ◇ "Evaluating tail call elimination in the face of return address protection, part 2"
- "What to do when you have a crash in the runtime control flow guard check" – Windows Control Flow Guard (CFG), узагальнення згаданого вище обмеження на місця, звідки відбуваються системні виклики, на інші API.

Додаткова література:

- http://en.wikipedia.org/wiki/Calling_convention
- http://en.wikipedia.org/wiki/X86_calling_conventions
http://en.wikipedia.org/wiki/Function_prologue
- <http://www.agner.org/optimize/> — "Calling conventions for different C++ compilers and operating systems" by Agner Fog, детальний, повний та доволі точний довідник.
- Вікі-книжка про дизасемблювання коду x86:
http://en.wikibooks.org/wiki/X86_Disassembly/The_Stack, http://en.wikibooks.org/wiki/X86_Disassembly/Functions_and_Stack_Frame_Examples,
http://en.wikibooks.org/wiki/X86_Disassembly/Calling_Conventions,
http://en.wikibooks.org/wiki/X86_Disassembly/Calling_Convention_Examples.
- <http://wiki.osdev.org/Stack> — розмова про стек процесорів сімейства x86 на сайті любителів розробки власних операційних систем.
- <http://www.unixwiz.net/techtips/win32-callconv-asm.html> і
<http://www.unixwiz.net/techtips/win32-callconv.html>. Увага, використано "діалект" асемблера від AT&T, зокрема, порядок аргументів зворотній до використаного вище.
- Приклад багу: <https://randomascii.wordpress.com/2022/12/14/compiler-tricks-to-avoid-abi-induced-crashes/> та
<https://randomascii.wordpress.com/2022/11/21/please-restore-our-registers-when-youre-done-with-them/>.

- <https://devblogs.microsoft.com/oldnewthing/20210901-00/?p=105632>
– "Looking at world through __stdcall-colored glasses" та <https://devblogs.microsoft.com/oldnewthing/20210902-00/?p=105639>
– "Adventures in application compatibility: The cost of forgetting to specify a calling convention".
- "Phrase origin: Why do we 'call' functions?" – історична довідка.

Кодування команд

- Кожна асемблерна інструкція (не плутати із директивами) перетворюється в машинні команди.
- Останні являють собою якісь послідовності бітів.
- У різних архітектурах можлива довжина команд відрізняється.
- x86, як архітектура архаїчна, та ще й CISC, має команди різної довжини, від 1 до 15 байт⁴⁴. Це проблема для декодера.
- Інші ISA, особливо RISC-процесорів, використовують фіксовану довжину (32 біти для ARM Cortex A) чи невеликий набір фіксованих варіантів (32 або 16 бітів для ARM Cortex M).
- Ще одним важливим аспектом є густина коду (code density) – наскільки ефективно ISA використовує біти для представлення команд. Особливо це важливо для MCU, із їх жорсткими обмеженнями по пам'яті програм. (Власне тому Cortex M має "скорочений" варіант команд, thumb-2).

⁴⁴Досягається екзотичними способами – в нормі таких довгих не буде

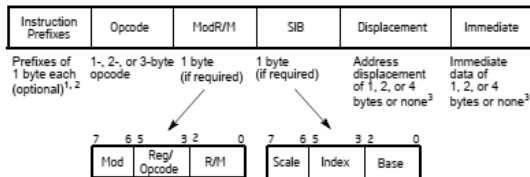
- Команда повинна "пояснити" процесору:
 - 1 Що слід зробити – Op code.
 - 2 Над якими аргументами: першим, другим, третім і т.д.
 - 3 Якщо використано регістр, слід помістити його номер.
 - 4 Якщо пам'ять – має бути закодовано спосіб взяти адресу.
 - 5 Безпосереднє значення – скажімо, константа для збереження в регістр чи зміщення для стрибка, теж повинно зберігатися в коді.
- Через природний інженерний конфлікт – хочеться якомога менші за розміром та максимально гнучкі команди, доводиться йти на компроміси.
- Наприклад, на ISA MIPS, команда має 32 біти, тому, для збереження безпосереднього значення виділяється 16 біт для цілочисельних арифметичних операцій⁴⁵, для переходів – 26 (ціною меншої кількості опкодів)⁴⁶.

⁴⁵Див. книгу Харріса і Паттерсона із списку літератури. Кому зовсім лінь – <http://www.cs.uwm.edu/classes/cs315/Bacon/Lecture/HTML/ch05s07.html>.

⁴⁶Як ви думаєте, чому, при тому, доступне зміщення – 2^{28} ?



- Кодування x86 складне і плутане, тому розглянемо лише декілька прикладів.



- ModR/M – дозволяє вибрати один із 8 регістрів або один із 24 режимів адресації. Решта полів використовуються в залежності від потреби.
- В 64-бітному режимі частину опкодів було використано по іншому.
- Детальніше див. <http://ref.x86asm.net> – список команд та <https://www-user.tu-chemnitz.de/~heha/viewchm.php/hs/x86.chm/x86.htm> – огляд.
- Приклади – див. `encodings_64.lst`!

Стилі коду

- Використання правильного стилю коду для асемблера чи не важливіше, ніж, для інших мов.
- ◇ Однак, через сукупність причин, популярних рекомендацій (guidelines) є мало.
- Декілька прикладів:
- ◇ https://projectacrn.github.io/latest/developer-guides/asm_coding_guidelines.html
- ◇ <http://www.sourceformat.com/coding-standard-asm.htm>
- ◇ <http://www.sourceformat.com/coding-standard-asm-style.htm>
- Структура традиційна – ліворуч мітки, можливо – на окремому рядку.
- Команди вирівняні однаково, незалежно від вкладеності.
- Нетипова для інших мов порада – точка входу в підпрограму повинна бути лише одна.
- ◇ Більш звична порада – одна точка повернення з функції, теж має більше сенсу, коли ви можете просто стрибнути за межі функції.

Префікси

- lock – 0xF0.
- REP/REPE/REPZ – 0xF3. REPNE/REPNZ – 0xF2.
- Segment override: 36, 26, 64, 65, 2E, 3E.
- Operand-size override prefix – 0x66
- Address-size override prefix – 0x67.
- REX prefix – 0x40H-0x4F, дозволяють в 64-бітному режимі вибрати додаткові регістри, вказати використання R-регістрів на противагу E-регістрам і т.д.

- Приклад довгого опкоду⁴⁷:

```
lock add dword ptr cs:[esi+8*edi+0x12345678], 0xABCCDEFCC
```

14 байт в 64-бітному режимі:

```
2e 67 f0 81 84 fe 78 56 34 12 cc ef cd ab
```

13 байт в 32-бітному режимі:

```
2e f0 81 84 fe 78 56 34 12 cc ef cd ab
```

- Студенти запропонували кращий:

```
hacquire lock add dword ptr cs:[esi+8*edi+0x12345678], 0xABCCDEFCC
```

```
2e 67 f2 f0 81 84 fe 78 56 34 12 cc ef cd ab
```

⁴⁷(Без чітерства із повторами префіксів – рівно по одному кожного, F0 – lock, 2E – сегмент cs). Ліміт 15 байт накладається апаратною, то ще байт можна додати.

Kernel-mode засоби x86

- Детально ми їх не розглядаємо⁴⁸ – лише основні ідеї.
- Для керування процесором та комп'ютерною системою в цілому, CPU містить привілейовані команди та системні регістри.
- Приклад таких регістрів:
 - ◇ CR0 – прапорці керування процесором.
 - ◇ CR3 – вказівник на (корінь ієрархії) таблиці сторінок.
 - ◇ Див. також CR2, CR4, CR8, XCR0.
 - ◇ Регістри для відлагодження: DR0-3, DR6, DR7.
 - ◇ GDTR, LDTR – (legacy) регістри таблиць сегментів.
 - ◇ IDTR – регістр, що вказує на таблицю переривань (див. далі).
 - ◇ TR – регістр дескриптора поточної задачі (теж legacy, але з сегмента задачі, TSS, на який вказує TR, наприклад, береться адреса стеку при переході в режим ядра).
 - ◇ Величезна кількість Model Specific registers (MSR) – від лічильника тактів, регістра адреси точки входу для SYSCALL, до керування кешуванням.

⁴⁸Трохи подробиць щодо 32-бітного режиму: <http://indrekis2.blogspot.com/2011/08/x86.html> — "Уроки захищеного режиму x86. Весь цикл разом", а список: https://en.wikipedia.org/wiki/Control_register.

I/O порти

- На сучасних ОС, користувацьким програмам звертатися до портів заборонено.
- Портів – 64K.
- Вивести в порт із номером 0-255, заданим безпосереднім значенням:
OUT imm_port8, AL/AX/EAX/RAX
- Номер порту – в DX.
OUT DX, AL/AX/EAX/RAX
- Аналогічно, читання з порту:
IN AL/AX/EAX/RAX, imm_port8
IN AL/AX/EAX/RAX, DX
- INS/INSB/INSW/INSD та OUTS/OUTSB/OUTSW/OUTSD – їх стрічкові аналоги, що беруть чи кладуть дані із/в [SI/ESI/RSI]/[DI/EDI/RDI] та збільшують або зменшують (в залежності від DF) відповідні регістри на 1/2/4/8. Дружать із префіксом REP.
- Див. також https://wiki.osdev.org/I/O_Ports.

I/O порти – приклад

- Можливості VGA відеокарти⁴⁹ досі є "спільним знаменником" відеокарт на персональних комп'ютерах із CPU x86, крім продуктів Apple – її інтерфейси все ще підтримуються для зворотної сумісності.
- ◇ Та й роз'єм відповідний, хоч і давно застарілий, ще можна зустріти:



- Комп'ютер взаємодіє із VGA як за допомогою memory mapped I/O, так і портів вводу-виводу.
- ◇ Більш модернові процесори не користуються підходом портів, тому ця стара система – хороший спосіб продемонструвати відповідну ідею⁵⁰.

⁴⁹Запроваджена в IBM PC PS/2, в 1987.

⁵⁰Як ми будемо не раз бачити, ідеї мають схильність повертатися, тому їх варто розуміти.

- У текстовому режимі ASCII-коди літер, що потрібно відобразити на екрані, зберігаються за парними адресами, починаючи із лінійної адреси 0xB8000⁵¹.
- ◇ За адресою 0xB8000 є символ, що відображається у лівому верхньому кутку екрану, потім – другий в верхньому рядку і т.д.
- Тобто, що відображати на екрані, задається memory mapped I/O.
- Керування курсором здійснюється за допомогою портів⁵².

⁵¹Насправді, це має ще давнішу історію, яку VGA успадкувала, для зворотної сумісності, від CGA. Непарні адреси містять атрибуту символу – колір символу, фону, мигання тощо. Детальніше див., наприклад, <http://indrekis2.blogspot.com/2019/04/0xb800.html>

⁵²Як і переключення режиму та багато чого ще, детальніше див. <http://www.osdever.net/FreeVGA/vga/vga.htm> та https://wiki.osdev.org/VGA_Hardware.

- Вивести стрічку за вказаними координатами, для типового режиму 80x25⁵³:

```
volatile unsigned char __far * const video_base =  
    (volatile unsigned char __far *) MK_FP( 0xB800, 0x0000);  
  
void put_string_at(int x, int y, int attr, const char *string){  
    int pos = 2 * ( x * 80 + y );  
    while( *string != 0){  
        *(video_base + pos++) = *string++;  
        *(video_base + pos++) = attr;  
    }  
}
```

- ◇ Синтаксис OpenWatcom 2, але щоб запустити цей код, буде потрібна DOS та відповідний емулятор, або достатньо старе "залізо".
- ◇ video_base – вказівник на 0xB8000, який створено згідно прийнятої в 8086 сегментної моделі⁵⁴.

⁵³Багато сучасних консолей користуються цим значенням за замовчуванням – ще один слід зворотної сумісності із технологіями 1980-х.

⁵⁴Ми коротко будемо розглядати її в майбутньому.

- Зробити курсор висотою у літеру:

```
void my_port_out(uint16_t port, uint8_t val);  
#pragma aux my_port_out \  
parm [DX] [AL] = \  
"out DX, AL"  
.....  
    my_port_out(0x3D4, 0x0A); // Set cursor upper scan line  
    my_port_out(0x3D5, 0x00); // Scan line value, counting from top
```

- Синтаксис OpenWatcom2 C тут сильно плутає, на асемблері той же код буде очевиднішим:

```
mov    EDX, 0x3D4    ; Port to set internal VGA card  
                        ; register address for next I/O  
mov    AL, 0x0A      ; Cursor conf. VGA I/O reg.  
out    DX, AL       ; Put 0x0A to I/O port 0x3D4  
  
mov    EDX, 0x3D5    ; Port to set the value to the VGA register  
                        ; chosen by the previous operation  
mov    AL, 0x00      ;  
out    DX, AL       ; Put 0x00 to I/O port 0x3D5
```

- Для економії адрес портів вводу-виводу, VGA відеокарта використовує невелику їх кількість, за допомогою опосередкованості та пар портів – в один записується номер внутрішнього регістра, тоді в інший пишеться, чи з нього читається значення того регістра.
- Малий адресний простір вводу виводу без можливості віртуалізації – одна із причин витіснення цього підходу memory mapped I/O.

Переривання

- Термін переривання – достатньо багатогранний. Розкручуватимемо його потроху.
- Базово: це сигнал процесору, зазвичай – від периферії, що є щось, що потребує його уваги.
- Такий запит називають Interrupt request, IRQ.
- У відповідь на цей сигнал, процесор зберігає свій стан, у мірі, достатній, щоб можна було продовжити виконання перерваного коду⁵⁵.
- Після цього, керування передається обробнику – Interrupt Service Routine (ISR).
- Це – апаратні переривання, Hardware interrupts.
- Вони – асинхронні, не синхронізовані із "внутрішнім життям" процесора.
- Контролер переривань може бути як внутрішнім (NVIC в ARM Cortex M), так і зовнішнім (APIC для x86).

⁵⁵Об'єм стану, що зберігається автоматично, сильно залежить від архітектури. x86 зберігає лише адресу повернення (far) та прапорці (про legacy interrupt gate не згадуємо), тому обробник переривань повинен зберегти і відновити все, що змінюватиме – стан процесора, регістри, так, щоб після повернення, перервана програма не відчувала відмінності. ARM зберігає все, що є caller-safe для його ABI, завдяки чому, обробник може бути C-функцією.

Пріоритет

- Часто, на час обробки переривання, інші переривання заборонені, в залежності від архітектури – безумовно, чи цим можна керувати (x86). При цьому, вони можуть як очікувати, щоб спрацювати після завершення ISR, так і губитися.
- Багато систем підтримують пріоритет переривань – більш пріоритетне, (таке, що стосується пристрою, на який потрібно реагувати швидше), може переривати виконання не тільки користувацького коду, але й ISR менш пріоритетного переривання.
- Підказка на майбутнє: ISR повинна бути якомога компактнішою. Якщо коротко – чим довше виконується, тим довше інші переривання залишатимуться без уваги та на довше буде зупинено перерваний для обробки код. Детальніше – в майбутньому.
- Часто підтримується спосіб тимчасово заборонити переривання – для виконання коду, що не можна переривати. Забороняти можна як всі переривання⁵⁶, так і лише деякі (зазвичай, за допомогою відповідного контролеру переривань).

⁵⁶Наприклад, команда cli (x86) очищає прапорець IF, забороняючи переривання. Навколо цього прапорця крутяться одне із джерел проблем віртуалізації x86. Команда POPF теж може його встановити чи очистити. Але, якщо виконання cli/sti кодом із недостатніми привілеями генеруватиме виключення, то POPF із непривілейованого коду просто ігноруватиме цей біт.

- Non-maskable interrupt (NMI) – різновид апаратного переривання, яке не можна заборонити.
- Зазвичай, використовуються для реакції на критичні помилки апаратури, на які слід реагувати негайно.
- Трапляються і більш екзотичні використання. Наприклад, для відладки. Колись використовувалося ширше, скажімо, для взаємодії із x87 на IBM PC чи й обробки клавіатури. Більше екзотичних історичних прикладів тут:
https://en.wikipedia.org/wiki/Non-maskable_interrupt.

Програмні переривання

- Крім апаратних переривань, існують програмні переривання – які або виникають всередині процесора, або генеруються спеціальною командою.
- Переривання, що генерує сам процесор, є реакцією на ті чи інші помилкові ситуації. Це може бути ділення на нуль чи відсутність сторінки, до якої відбулося звертання, в пам'яті. Їх називають виключеннями (exception) або пастками (trap)⁵⁷.
- Такі переривання – синхронні.

⁵⁷x86 має хитру ієрархію таких переривань. Exception буває trap, fault або abort. Trap спрацьовує зразу після завершення команди, що її породила, IP вказує на наступну інструкцію. Це, наприклад, int 0x04, Overflow. Fault – IP вказуватиме на саму команду, ідея таких переривань – виправити помилкову ситуацію і продовжити виконання. Це, наприклад, int 0x0E, Page Fault. Abort – помилка, відновитися, після якої неможливо, наприклад, int 0x08, Double Fault. Детальніше див. <https://wiki.osdev.org/Exceptions> та <http://indrekis2.blogspot.com/2011/08/x86.html> – урок 4.

- Програмне переривання також може спричинити спеціальна команда, наприклад, для x86:

```
int 21h      ; Чи так: int 0x21
```

- Для ARM Cortex M, з його NVIC, все трохи веселіше – є спеціальний регістр, STIR. Запис в нього номера переривання, генерує відповідне IRQ:

```
mov    R0, #0x5      ; Номер переривання
ldr    R1, =0xE000EF00 ; адреса NVIC STIR
str    R0, [R1]      ; Увага, записуємо R0 --> [R1]
dsb                                ; Бар'єр синхронізації -- переривання
                                ; може виникнути не негайно.
```

Зауважте, синхронним воно не буде – відбувається через зовнішній контролер переривань, як зовнішнє переривання.

Вектори переривань

- Переривання⁵⁸ мають свій номер. Зазвичай, це однобайтове число, від 0 до 255⁵⁹.
- Обробник переривання задається так званим **вектором переривання** – адресою обробника у спеціальній таблиці⁶⁰.
- Таблиця ця може бути фіксованою в пам'яті⁶¹, або її адреса задається спеціальним регістром⁶².
- Однак, тут є важливе джерело плутанини. Апаратні запити на переривання, IRQ, мають свої, **незалежні** номери!

⁵⁸Зазвичай – які тільки екзотичні архітектури не трапляються... Не буду більше повторювати це.

⁵⁹Реалізованими в залізі можуть бути не всі, навіть якщо архітектура підтримує. ARM Cortex M підтримує до 240 векторів.

⁶⁰IDT – Interrupt descriptor table на x86, ARM Cortex M просто каже Interrupt vector table, IVT

⁶¹Починається із нульової адреси на x86 в реальному режимі та ARM Cortex M за замовчуванням.

⁶²IDTR на x86, VTOR (Vector Table Offset Register) для ARM Cortex M.

- Однак, тут є важливе джерело плутанини. Апаратні запити на переривання, IRQ, мають свої, **незалежні** номери!
- Наприклад, архітектура IBM PC AT+, працюючи в реальному режимі, відображає IRQ0-IRQ7 на int 0x08-0x0F, а IRQ8-IRQ15 на int 0x70-0x77. В захищеному режимі ОС мусить змінювати це відображення, оскільки перших 32 вектори переривань зарезервовані для процесора.
- ◇ ARM Cortex M теж не без нюансів. Переривання від процесора мають від'ємні номери IRQ, таких є 16, і ще 240 – зовнішніх.
- Тобто, як тільки ваш не-Apple комп'ютер ввімкнувся, якщо він підтримує сумісність із BIOS, (не є виключно UEFI системою), то IRQ0 – переривання від таймера, яке викличе обробник із вектором 7, еквівалент виклику int 0x07.
- Для x86 обробники переривань зараз пишуть не часто. Тому на наступному слайді приклад із сивої давнини – сповільнюємо виконання програми, не заважаючи роботі таймера⁶³.
- ◇ А ось для ARM Cortex M іноді доводиться, тому ще нижче – приклад, який нам стане в пригоді у майбутньому.

⁶³Взято із книги Тома Свана про турбо-асемблер, тому синтаксис – TASM.

```

PROC    SlowMo
    cmp    [byte cs:InProgress], 0 ; Check in-progress flag
    jne    @@99                    ; Jump if SlowMo is running
    inc    [byte cs:InProgress]    ; Else, set flag = 1
    sti                                ; Allow interrupts to occur
    push   ax                      ; Save modified registers
    push   ds
    push   dx
    mov    al, EOI                  ; al = end-of-interrupt value
    out    PIC8259, al              ; Issue end of interrupt
    mov    ax, BIOSData            ; Address BIOS data area
    mov    ds, ax                  ; with ds
    mov    ax, [word LowTimer]     ; Get low word of timer value
@@10:   mov    dx, [word LowTimer] ; Get new timer value into dx
    sub    dx, ax                  ; Subtract new-old timer
    cmp    dx, [cs:difference]     ; Compare to difference
    jb     @@10                    ; Loop until difference passes

    cli                                ; Disable interrupts
    dec    [byte cs:InProgress]    ; Reset in-progress flag
    pop    dx                      ; Restore saved registers
    pop    ds
    pop    ax
@@99:   ired                      ; Interrupt return

```

- Хоча переривання ARM Cortex M зберігають стан, достатній для використання в ролі обробника звичайної функції C, іноді в процес хочеться втрутитися.
- Тоді допоможе асемблер.
- Цей приклад розглядатиметься в майбутньому. Ціль – дозволити програмі, що користується semihosting, спеціальним інтерфейсом відлагодження цих мікроконтролерів, працювати і за відсутності дебагера. При цьому, коду на C потрібен доступ до кадру стеку із збереженими регістрами.

HardFault_Handler:

```
tst    LR, #4
ite    eq
mrseq  R0, MSP    /* stacking was using MSP */
mrsne  R0, PSP    /* stacking was using PSP */
mov    R1, LR     /* second parameter */
ldr    R2,=HardFault_Handler_C
bx     R2
```

де HardFault_Handler_C – функція на C, із прототипом:

```
void HardFault_Handler_C(exception_stack_frame_t* frame,
                          unsigned int lr_value);
```

- Детальніше див. <http://indrekis2.blogspot.com/2017/12/semihosting.html>.

Екзотичні команди переривань x86

- Команда INT 3 має однобайтовий синонім, 0xCC (існує і двобайтовий 0xCD 0x03). Це переривання відлагодження. Щоб створити breakpoint, дебагер може замінити перший байт команди на 0xCC, та перехопити керування, коли виконання дійде до неї, відновивши правильний байт після. Детальніше див., наприклад, "[How debuggers work: Part 2 - Breakpoints](#)", "[Debugger breakpoints are usually implemented by patching the in-memory copy of the code](#)".
- INT 1 – переривання, яке відбувається, (автоматично) після кожної машинної команди, дозволяючи покрокове відлагодження. Перехід в цей режим здійснюється встановленням прапорця Trap Flag (TF) в EFLAGS. Див. https://en.wikipedia.org/wiki/Trap_flag. (Є також однобайтовий варіант, 0xF1, іноді використовують мнемоніку ICEBP).
- Із екзотики, INTO – генерувати переривання 0x00, якщо OF, BOUND – перевірка на вихід за межі масиву, переривання 0x05.
- Перераховані вище інструкції в прикладному ПЗ не особливо корисні, крім, може, INT3.
- ◇ Однак, сучасні процесори володіють значно просунутішими методами відлагодження. Вони керуються регістрами відлагодження, DR0...DR7, та набором MSR регістрів.
- ◇ Зокрема, DR0...DR3 дозволяють встановити чотири точки зупинки, в кодї чи **за доступом до даних**, решта – керують процесом.
- ◇ Є навіть можливість breakpoint по доступу до портів вводу-виводу.

Код, що сам себе модифікує (draft)

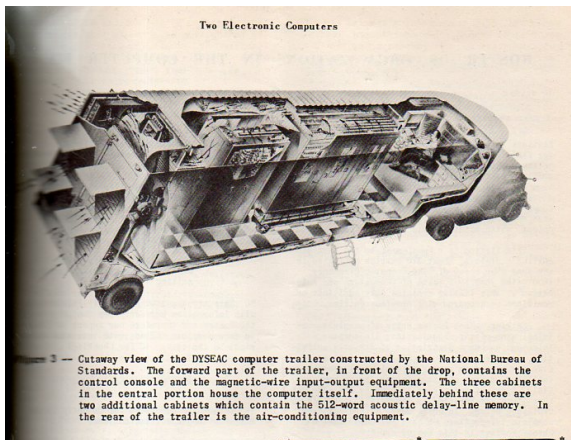
- Код, що сам себе модифікує, був дуже популярним трюком, заради економії – пам'яті, тактів і т.д.
- ◇ Процесор 6502 (зокрема, CPU Atari 2600):
<https://retrocomputing.stackexchange.com/questions/11108/why-does-the-6502-have-the-bit-instruction/11132#11132>.
- ◇ Для Altair 8800 (CPU 8080) в MS BASIC, написаному Білом Гейтсом (1975, перший продукт Microsoft):
<https://devblogs.microsoft.com/oldnewthing/20220111-00/?p=106144>, "Jumping into the middle of an instruction is not as strange as it sounds".
- ◇ Як на мене, дещо безумний спосіб завантажувати програму:
<http://www.righto.com/2021/02/an-ibm-1401-mainframe-computer-at.html>.
- До Pentium в x86 була програмно видима черга передвибірки – зміна команди, яка вже попала туди, не впливала на команду, що виконується. Pentium і новіші стежать за тим, але мусять скидати різноманітні внутрішні буфери.
- ◇ Всі ці труднощі сповільнюють і створюють незручності.
- ◇ Інші сучасні процесори можуть мати значно складніші відносини із кодом, що сам себе модифікує.
- В будь-якому випадку, зараз це швидше сприймається як дія вірусу.
- ◇ Щоправда, динамічний лінкер змушений модифікувати код – див. GOT, PLT і суміжні теми.

Посилання

- Пошук недокументованих інструкцій сучасних процесорів:
<https://github.com/xoreaxeaxeax/sandsifter>.
- Альтернативний підхід: <https://blog.can.ac/2021/03/22/speculating-x86-64-isa-with-one-weird-trick/>.
- Огляд проблеми недокументованих інструкцій для x86:
<https://www.cattius.com/images/undocumented-cpu-behavior.pdf>.
- Використання сегментних регістрів для приховування руткітів:
<https://blog.can.ac/2018/04/26/splitting-data-from-code-forgotten-x86-feature-segmentation-n/>:
використання для зловмисного (чи й навпаки – захисного) коду іншого сегменту коду, із своїм записом в GDT.
- ◇ Більше таких технологій тут: <http://www.phrack.org/issues/63/8.html> – “Raising The Bar For Windows Rootkit Detection”.
- Історичні приклади недокументованих інструкцій – для зовсім простих процесорів:
https://www.cpcwiki.eu/index.php/Z80_-_undocumented_opcodes,
<https://www.pagetable.com/?p=39> – “How MOS 6502 Illegal Opcodes really work”,
https://www.nesdev.org/wiki/CPU_unofficial_opcodes.
- ◇ 286: <https://rep-lodsb.mataroa.blog/blog/intel-286-secrets-ice-mode-and-f1-0f-04/>,
<https://rep-lodsb.mataroa.blog/blog/the-286s-internal-registers/>.
- Реалізація переривань в 8086:
<http://www.righto.com/2023/02/8086-interrupt.html>.
- <http://www.cap-lore.com/Hardware/int.html> – “History of Interrupts”, 2002.
- <https://people.computing.clemson.edu/~mark/interrupts.html> – “Interrupts”, Mark Smotherman, 2017.

DYSEAC

- Вікіпедія згадує (<https://en.wikipedia.org/wiki/DYSEAC>), що вперше переривання були реалізовані на комп'ютері DYSEAC, 1954. Я б полінувався про нього згадувати, але коли перевіряв дане твердження, натрапив на цікаву інформацію: "It was housed in a truck, making it one of the first portable computers (perhaps the first)". Всього 18 тон важив, ділив-множив всього лиш 2 мілісекунди (і майже стільки ж йшло на звертання до пам'яті).



Delay slot

- x86 та ARM такого не роблять, але в деяких платформах, наприклад, старіші варіанти MIPS та SPARC, існує поняття/явище delay slot.
- Як оптимізація (чи, швидше, спрощення реалізації) конвеєризації, інструкція, що йде зразу за командою переходу, виконується в будь-якому випадку.
- Аналогічно, на деяких архітектурах (старі MIPS) є load delay slot – команда після команди завантаження, ще не бачить результату.
- Приклад із wiki (https://en.wikipedia.org/wiki/Delay_slot):

```
lw    v0,4(v1)    # load word from address v1+4 into v0
nop                    # wasted load delay slot
jr    v0          # jump to the address specified by v0
nop                    # wasted branch delay slot
```

- Для сучасних реалізацій ця оптимізація не актуальна, але, для зворотної сумісності, її доводиться відтворювати.
- Подробиці – див. книгу Паттерсона і Хеннесі із списку літератури.

Системні виклики x86

Див. `os_pok_syscalls.pdf`

Список номерів системних викликів Linux:
<https://filippo.io/linux-syscall-table/>.

Calling convention for 64-bit Linux, BSD, MacOS

- Угоди про виклики для системних викликів та для викликів звичайних функцій – це різні поняття.
- ◇ Однак, часто вони дуже схожі – це раціонально, спрощує створення ядра ОС.
- Розгляд їх почнемо із 64-бітових варіантів Linux, BSD та MacOS.
- Аргументи передаються через регістри, в такому порядку: RDI, RSI, RDX, RCX, R8, R9, зліва направо; решта – через стек, справа наліво.
- Стек очищає функція, що здійснила виклик (caller).
- ◇ Для системних викликів, замість RCX використовується R10. Причина в тому⁶⁴, що команда syscall при вході в ядро, зберігає RIP в RCX, а RFLAGS в R11.
- Floating point – через XMM0-7 (чи їх більші варіанти – YMM, ZMM). Якщо функція із змінною кількістю аргументів (variadic), їхню кількість слід передати в AL (краще використовувати цілий RAX).

⁶⁴ Детальніше див. <https://stackoverflow.com/questions/47983371/why-do-x86-64-linux-system-calls-modify-rcx-and-what-does-the-value-mean> та <https://stackoverflow.com/questions/38577609/difference-in-abi-between-x86-64-linux-functions-and-syscalls>.

- Стек повинен бути вирівняний на границю параграфа, 16 байт.
- Результат – в RAX, якщо мало – ще RDX. Floating point – у ST(0) чи XMM0.
- Оскільки в функції стек завжди вирівняний, після виклику він буде вирівняний на 8, але не на 16 – треба виправляти до будь-якого виклику, бо функції можуть на це покладатися.
- Функція має відновити, якщо змінювала (callee-save) регістри: RBX, RBP, R12-R15.
- Може вільно змінювати (scratch) регістри: RAX, RCX, RDX, RSI, RDI, R8-R11, також: STx, XMMxx – всі.
- Існує Red zone – 128 байт **під стеком**, від [RSP-128] до [RSP-8], можна використовувати для тимчасових змінних, їх не затре обробник сигналу чи переривання. Однак, call чи push, природно, затиратиме.
- ◇ Детальніше про Red Zone, навіщо вона і хто ще може затерти стекову пам'ять, якщо ваша програма цього не робить (не викликає ф-цій і т.д.) – див.
<https://devblogs.microsoft.com/oldnewthing/?p=100685>, "Why do we even need to define a red zone? Can't I just use my stack for anything?"

Приклад – 64-бітовий Linux, libc, PIC

```
.text
main:
    push    rbp                # Stack should be aligned to 16!

    call    getpid
    mov     esi, eax           # Результат виклику -- в EAX
    lea     rdi, [RIP+msg]
                                # Lea а не mov, оскільки це лаконічніше.
    xor     rax, rax           # Кількість використаних XMM регістрів
                                # для variadic arg
    call    printf             # "printf rdi, rsi"

    xor     edi, edi           # exit code
    call    exit               # "exit (0)" ==> "exit edi"

.data
msg: .asciz "Current process ID is %d.\n"
```

- Код із прикладу hello_linux64_libc.s.

Calling conventions for 64-bit Windows

- Результат – в RAX цілий, ST(0) або XMM0 – FP.
- Аргументи передаються через RCX/XMM0, RDX/XMM1, R8/XMM2, R9/XMM3, (замість XMMx може бути також YMMx чи ZMMx), решта – через стек, справа наліво.
- Увага: аргументи можна присвоювати строго в такому порядку! Якщо перший аргумент floating point, пішов в XMM0, то другий аргумент, (якщо вміститься) – піде в RDX. Тобто, через регістри всього можна передати 4 аргументи (тоді як в підході Linux – $14 = 6 \text{ цілих} + 8 \text{ FP}$).
- Якщо floating point аргумент передається variadic-функції, типу printf(), його слід передавати і у відповідному XMM і в регістрі загального використання.
- ◇ Ймовірно, причина – спрощення підтримки функцій без прототипу (застарілий спосіб із K&R C), чи, менш ймовірно, variadic-функцій.
- ◇ Якщо компілятор не знає, як аргумент буде використано в функції, як цілий чи як FP, він зберігає його і в Rxx і в XMMx⁶⁵.

⁶⁵ Див., наприклад, <https://devblogs.microsoft.com/oldnewthing/20190820-00/?p=102792>.

Calling conventions for 64-bit Windows

- Стек має бути вирівняний на 16.
- Крім того, перед входом має бути зарезервовано в стеці 32 байти – shadow space.
- Функція не має права змінювати (callee-save) регістри: RBX, RSI, RDI, RBP, R12-R15, XMM6-XMM15 (але це не стосується їх старших частин для [Y|Z]MM).
- Може змінювати (scratch) регістри: RAX, RCX, RDX, R8-R11, STx, XMM0-XMM5, всі YMM/ZMM, крім нижніх частин XMM6-XMM15.
- Стек чистить той, хто викликав функцію.
- Хоча ця угода мала бути універсальною, для оптимізації потім додали `__vectorcall`: <https://docs.microsoft.com/en-us/cpp/cpp/vectorcall?view=msvc-160>. Використовується вона лише явно.

Приклад – 64-бітовий Windows, libc, лінуємо "самостійно"

```

.....
.text

main:
    push    rbp                # Stack should be aligned to 16!

    xor     ecx, ecx
    lea     rdx, [rip + OFFSET msg]
    lea     r8, [rip + OFFSET title]
    xor     r9d, r9d           # MB_OK == 0
    call    QWORD PTR [rip + __imp_MessageBoxA]
# Call to MessageBoxA( HWND_DESKTOP, msg, title, MB_OK );

    xor     ecx, ecx
    call    QWORD PTR [rip + __imp_ExitProcess]
# ExitProcess(0);

.data
msg:  .ascii "Win64 Assembly\0"
title: .ascii "Hi! I'm the example program!\0"

```

- Код із прикладу hello win64 gui min.s.

Обробка виключень

- Windows виділяє leaf-функції – такі, що можуть обмежитися роботою із scratch-регістрами, зарезервованою в стеку областю для параметрів (home space), **не змінюють регістри, що потрібно відновити та вказівник стеку**, а адреса повернення завжди залишається на вершині стеку.
- Причина виділення таких функцій – для всіх решту потрібно створювати коди розкрутки виключень, що зробити вручну на асемблері – нетривіально.
- Вимоги створювати ці коди не стосуються leaf-функцій.
- Детальніше див.
<https://docs.microsoft.com/en-us/cpp/build/exception-handling-x64>.
- Тобто, функція, що не налаштовує таблиць виключень і не є leaf-функцією, є, в загальному випадку, некоректною під Windows-64, хоча проблеми проявлятимуться лише під час обробки виключних ситуацій⁶⁶.
- MASM⁶⁷ надає спеціальні макроси для вирішення цієї проблеми – див. "Unwind helpers for MASM" за посиланням вище.
- Відправною точкою для GAS може бути код, згенерований компілятором, який містить відповідні директиви.
- Офіційна інформація про ці угоди про виклики:
<https://docs.microsoft.com/en-us/cpp/build/x64-software-conventions>.

⁶⁶ Мова про structured exceptions Windows, не лише про виключення C++.

⁶⁷ Асемблер від Microsoft.

Calling convention for 32-bit Linux, BSD, Mac

- Угод про виклики для 32-бітного коду дуже багато – історично та внаслідок спроб оптимізації різних виробників, що конкурували.
- Розглядаємо ті, що підтримує GCC.
- Параметри виключно через стек, справа наліво.
- Стек бажано вирівнювати на 16.
- Результат – в EAX. Floating point – у ST(0) чи XMM0.
- Функція не має права змінювати (callee-save) регістри: EBX, ESI, EDI, EBP.
- Може змінювати (scratch) регістри: EAX, ECX, EDX, також: STx, XMMx – всі.
- Приклад див. `hello_linux32_libc_pic.32.s`.

Calling conventions for 32-bit Windows

- Візуально, все схоже. Однак, 32-бітні угоди про виклики значно менш впорядковані!
- Результат: EAX, EDX, ST(0), XMM0 (і більші його варіанти).
- Функція не має права змінювати (callee-save) регістри: EBX, ESI, EDI, EBP.
- Може змінювати (scratch) регістри: EAX, ECX, EDX, ST_x, XMM_x.
- А тепер про зоопарк:
- **cdecl**: аргументи в стеку, справа наліво, стек чистить той, хто викликав⁶⁸. Приклад – libc/msvcrt.
- **stdcall**: те ж, але очищає стек сама функція. Приклад – функції Win32 API (крім vararg-функцій).
- **pascal**: аргументи в стеку, зліва направо, чистить – функція.
- **fastcall** від Microsoft⁶⁹: ECX, EDX, решта через стек, справа на ліво. Очищає функція.
- GNU/GCC fastcall робить схоже, різниця – в "return pointer".

⁶⁸GNU/GCC варіант трішки відрізняється – "return pointer" (вказівник, куди класти результат, якщо в виділені для нього регістри він не помістився, див. книгу Агнера Фога нижче) функція має очищати самостійно.

⁶⁹Intel наслідує нативний компілятор – MSVC чи GCC, в залежності від ОС. Borland чи Watcom теж мають свої варіанти. Значимість цих компіляторів зараз незначна, тому не деталізуватимемо.

- **thiscall** від Microsoft: ECX, решта через стек, справа наліво, очищає функція. Призначений для виклику членів класу.
- **this** неявно йде першим або другим параметром. (Подробиці – в Агнера Фога:
https://www.agner.org/optimize/calling_conventions.pdf).
- Багато компіляторів підтримують нестандартні розширення, які дозволяють уточнити угоди про виклики для даної функції:
`__cdecl`, `__stdcall`, `__pascal` і т.д.

```
int __stdcall SomeFunction (double a);
```

- Приклади: `hello_win32_con_o.asm`,
`hello_win32_con_selfimport.asm`,
`hello_win32_gui_selfimportPE.asm`

Загальні нотатки щодо угод про виклики

- Арифметичні прапорці можна змінювати довільно.
- Прапорець напрямку, DF вважається очищеним, якщо було встановлено для потреб функції – слід очистити потім.
- За потреби – якщо це дає помітний виграш у швидкодії абоощо, в ізольованих блоках коду можна відхилитися від даних угод для вашого власного коду.
- В тексті вище практично не зупинялися на роботі із floating-point та нетривіальними об'єктами – великими, чи із конструкторами, віртуальними функціями тощо. Також, не розглядався name mangling і багато інших технічних подробиць. Детальніше див. agner.org/optimize/ – "Software optimization resources" від Агнера Фора, том 5-й, "Calling conventions for different C++ compilers and operating systems". Вимоги до формування кадру стеку можуть бути важливими, але не розглядаються і в нього.
- Intel, для своєї (невдалої) спроби створити принципово новий 64-бітовий процесор – Itanium, розробила ABI – calling conventions і т.д., для коду нового процесора, щоб уникнути хаосу з ними, який характерний для x86. Цей ABI почав використовувати GCC, потім Clang – чому ні, якщо вже хтось проєкту роботу виконав. Таким чином, хоча Ітаніуми мертві, ABI, яке GCC та Clang використовують, називається Itanium ABI.
- Додаткові посилання: https://en.wikipedia.org/wiki/X86_calling_conventions, https://en.wikibooks.org/wiki/X86_Disassembly/Calling_Conventions, <https://www.codeproject.com/Articles/1388/Calling-Conventions-Demystified>.
- "What is a static chain pointer in the context of calling convention ABI?", "The mysterious second parameter to the x86 ENTER instruction", "Lexically scoped functions accessing parent locals: The display", для мов з вкладеними функціями.

Посилання

- Особливості деяких ISA показують, чому певні дії є невизначеною поведінкою в C/C++. До прикладу: "[On how different Windows ABIs choose how to pass 32-bit values in 64-bit registers](#)" та "[The Alpha AXP, part 16: What are the dire consequences of having 32-bit values in non-canonical form?](#)".

Статичне та динамічне лінування, PIC, PIE, і "фіксований" код

- Під час лінування лінкер встановлює зв'язки між звертанням до глобальних⁷⁰ змінних та їх розташуванням у пам'яті, місцем виклику функцій та їх розташуванням у пам'яті процесу тощо.
- При цьому можуть створюватися так звані релокації (relocations) – місця, де потрібно підправити адресу після завантаження у пам'ять.
- ◇ До прикладу: відомо зміщення змінної чи функції відносно початку образу програми в пам'яті; завантажувач (loader) завантажує цей образ за певною адресою, і тоді до зміщення додає адресу початку образу. Для цього завантажувач модифікує команду безпосередньо в пам'яті. Щоб знайти ці місця, виконавчий файл містить список релокацій – їхні адреси та зміщення.
- Статичне лінування – всі адреси розв'язуються під час лінування.
- ◇ Якщо компілюється для завантаження за фіксованою адресою, не міститиме релокацій, інакше – все ще може їх містити.

⁷⁰Та близьких до них технічно змінних рівня просторів імен, локальних статичних змінних тощо. ▶ ◀ ≡ ≡ ≡ ≡ ≡ ≡ ≡ ≡ ≡

Position Independent Code

- Альтернативний підхід до використання релокацій – **PIE**, Position Independent Code.
- ◇ Код компілюється так, щоб не залежати від того, де в пам'яті його завантажать.
- ◇ Суміжне поняття – **PIE**, Position Independent Executable. Такий виконавчий файл мусить використовувати лише динамічні бібліотеки, його розташування в пам'яті не фіксується.
- ◇◇ Основна різниця, PIE-бібліотеки можна перехоплювати (interpose⁷¹, а PIE – ні).
- Не-PIE програма завжди завантажеться за адресою, вказаною лінкером.

⁷¹Див. <http://www.drdoobbs.com/building-library-interposers-for-fun-and/184404926> або https://web.archive.org/web/20120626005255/http://developers.sun.com/solaris/articles/lib_interposers.html

Position Independent Code та динамічне зв'язування

- Динамічне лінування – зв'язування із функціями, змінними тощо із динамічних бібліотек, відбувається під час завантаження та лінування програми.
- ◇ Хоча воно можливе з використанням релокацій, (і деякі системи так і роблять) це неефективно – потребує модифікації сторінок із кодом, та не дає їх повторно використовувати у різних процесах, оскільки співпадіння адрес для релокацій – малоімовірне.
- ◇ Частіше використовується PIC-підхід, разом із спеціальними структурами (GOT і PLT, див. далі), у яких відбувається релокація.
- "Exploring the section layout in linker output" – про секції, сегменти і як тим всім керувати.

- Розглянемо на прикладі простої програми на C, із двох частин, з глобальною змінною та функцією в одному модулі (об'єктному файлі) та звертанням до них із іншого модуля.
- Теоретичні основи всієї цієї машинерії описані в "Linkers and Loaders" із списку літератури, а подробиці – в документації на конкретні системи.
- Хороша коротка стаття: "[PLT and GOT - the key to code sharing and dynamic libraries](#)".
- Серія більш детальних статей: "[How statically linked programs run on Linux](#)", "[Load-time relocation of shared libraries](#)", "[Position Independent Code \(PIC\) in shared libraries](#)", "[Position Independent Code \(PIC\) in shared libraries on x64](#)".
- Аналогічна серія для MS-Windows (16-і 32-бітового): "[Index to the series on DLL imports and exports](#)".
- Його ж: "[Why does the debugger show me the wrong function?](#)", "[Why is Identical COMDAT Folding called Identical COMDAT Folding?](#)", "[How can I confirm in the Windows debugger that I'm looking at a COMDAT-folded function?](#)"
- Серія тематичних есеїв від того ж автора: "[Using the classical model for linking to provide unit test overrides](#)", "[Understanding the classical model for linking: You can override an LIB with another LIB, and a LIB with an OBJ, but you can't override an OBJ](#)", "[Understanding the classical model for linking, groundwork: The algorithm](#)", "[Understanding the classical model for linking: Taking symbols along for the ride](#)", "".

- Див. також серію статей із оглядом особливостей процесу компілювання від Fabien Sanglard⁷²:
 - ◇ <https://fabiensanglard.net/dc/index.php> – “Introduction”.
 - ◇ <https://fabiensanglard.net/dc/driver.php> – “The Compiler Driver”, стадії компіляції, від препроцесингу до лінування.
 - ◇ <https://fabiensanglard.net/dc/cpp.php> – “The Preprocessor”, зокрема про precompiled headers,
 - ◇ <https://fabiensanglard.net/dc/cc.php> – “The Compiler”, зокрема структура: frontend-optimizer-backend.
 - ◇ <https://fabiensanglard.net/dc/ld.php> – “The Linker”, зокрема згадує скрипти лінкера,
 - ◇ <https://fabiensanglard.net/dc/loader.php> – “The Loader”, про те, як відбувається завантаження процесу.
- <https://reverseengineering.stackexchange.com/a/1993> – список статей від Ian Lance Taylor, автора лінкера GOLD, про роботу лінкера, релокації тощо.
- Деякі аспекти ми також зустрінемо в розмовах про асемблер.

⁷²Зацитую автора: “A contrasting experience was to learn how to use the tools to turn my programs into executable. It was a painfully slow and deeply unpleasant process where knowledge was gathered here and there after trial, errors, and a lot of time spent on search engines. It felt like acquiring the same level of comfort to use a compiler took significantly more than learning the language.”.

Експериментальна програма

- library.c

```
#include <stdio.h>
```

```
#include "file1.h"
```

```
int global_var; // Глобальна змінна
```

```
void library_function(int arg){    // Функція  
    printf("Argument: %i\n", arg); // Виклик зовнішньої ф-ції  
    // Звертання до глобальної змінної із того ж модуля,  
    // де вона визначена  
    printf("Global var: %i\n", global_var);  
    global_var = arg;  
}
```

- file1.h

```
#ifndef FILE1_HEADER
```

```
#define FILE1_HEADER
```

```
extern int global_var;
```

```
void library_function(int arg);
```

- main_prog.c

```
#include <stdio.h>
```

```
#include "file1.h"
```

```
void local_function(){  
    printf("Local function called\n");  
}
```

```
int main() {  
    // Звертання до глобальної змінної із іншого модуля:  
    printf("Global value before: %i", global_var);  
    global_var = 13;  
    printf("Global value after: %i", global_var);  
  
    // Виклик функції із іншого модуля:  
    library_function(42);  
  
    // Виклик функції із цього модуля:  
    local_function();  
  
    return 0;  
}
```

Статичне по-PIE лінування

- Розпочнемо із найпрямолінійнішого варіанту – повністю статичне по-PIE лінування:

```
gcc -c -O0 -g -static -fno-pic main_prog.c -o main_prog.o
gcc -c -O0 -g -static -fno-pic library.c -o library.o
ar rcs liblibrary.a library.o
gcc -O0 -g -static -fno-pic main_prog.o liblibrary.a -o main_prog \
    -llibrary -L.
```

- Створені файли:

```
$ ls -l main_prog *.o -X -r | tr -s ' ' | cut -d" " -f 5,9
4664  main_prog.o
4192  library.o
902520 main_prog
```

```
$ size library.o # Зверніть увагу на розмір BSS:
```

text	data	bss	dec	hex	filename
188	0	4	192	c0	library.o

```
$ size main_prog.o
```

text	data	bss	dec	hex	filename
301	0	0	301	12d	main_prog.o

```
$ size main_prog
```

text	data	bss	dec	hex	filename
782466	23232	23016	828714	ca52a	main_prog

```
$ ldd ./main_prog
```

```
not a dynamic executable
```

Статичний виконавчий файл

- Бачимо, що точка входу 0x401650:

```
$ objdump -f main_prog
```

```
main_prog:      file format elf64-x86-64
architecture: i386:x86-64, flags 0x00000112:
EXEC_P, HAS_SYMS, D_PAGED
start address 0x0000000000401650
```

- Подивимося на релокації.
 - У виконавчому файлі їх, природно, немає.
- ```
$ objdump -r --section=.text main_prog
```

```
main_prog: file format elf64-x86-64
```

- Всі символи можна побачити за допомогою опції -t, але повний вивід буде занадто великим:

```
$ objdump -t main_prog | grep global_var
00000000004c72f0 g 0 .bss 0000000000000004 global_var
$ objdump -t main_prog | grep local_function
0000000000401775 g F .text 0000000000000015 local_function
$ objdump -t main_prog | grep library_function
00000000004017e5 g F .text 0000000000000046 library_function
```

- Об'єктний файл (із) бібліотеки:

```
$ objdump -r --section=.text library.o
```

```
library.o: file format elf64-x86-64
```

```
RELOCATION RECORDS FOR [.text]:
```

| OFFSET           | TYPE           | VALUE                         |
|------------------|----------------|-------------------------------|
| 0000000000000015 | R_X86_64_32    | .rodata                       |
| 000000000000001f | R_X86_64_PLT32 | printf-0x0000000000000004     |
| 0000000000000025 | R_X86_64_PC32  | global_var-0x0000000000000004 |
| 000000000000002c | R_X86_64_32    | .rodata+0x000000000000000e    |
| 0000000000000036 | R_X86_64_PLT32 | printf-0x0000000000000004     |
| 000000000000003f | R_X86_64_PC32  | global_var-0x0000000000000004 |

- Зауважте `global_var` – дві релокації, за адресами 25 і 3F (виду `R_X86_64_PC32`) та `R_X86_64_32` – можна запідозрити, що це перше місця першого і другого звертання до цієї змінної.
- Також, дві релокації для `printf` (типу `R_X86_64_PLT32` – див. далі) – 1F та 36.
- Релокації у `.rodata` – текстові стрічки, зі зміщенням `0x00` і `0x0E`<sup>73</sup> (див. наступний слайд).

<sup>73</sup>Якщо подивитися на першу із стрічок, вона має якраз `14 = 0xE` байт

- Символи із бібліотеки (двома різними способами):

```
$ objdump -t library.o
```

```
library.o: file format elf64-x86-64
```

```
SYMBOL TABLE:
```

```
0000000000000000 l df *ABS* 0000000000000000 library.c
0000000000000000 l d .text 0000000000000000 .text
0000000000000000 l d .rodata 0000000000000000 .rodata
0000000000000000 l d .debug_info 0000000000000000 .debug_info
0000000000000000 l d .debug_abbrev 0000000000000000 .debug_abbrev
0000000000000000 l d .debug_line 0000000000000000 .debug_line
0000000000000000 l d .debug_str 0000000000000000 .debug_str
0000000000000000 l d .debug_line_str 0000000000000000 .debug_line_str
0000000000000000 g 0 .bss 0000000000000004 global_var
0000000000000000 g F .text 0000000000000046 library_function
0000000000000000 *UND* 0000000000000000 printf
```

```
$ nm -C library.o
```

```
0000000000000000 B global_var
0000000000000000 T library_function
 U printf
```

- Секції даних:

```
$ objdump -s --section=.data library.o # Порожня
```

```
$ objdump -s --section=.rodata library.o
```

```
.....
```

```
Contents of section .rodata:
```

```
0000 41726775 6d656e74 3a202569 0a00476c Argument: %i..Gl
0010 6f62616c 20766172 3a202569 0a00 obal var: %i..
```

- Файл із функцією main:

```
$ objdump -r --section=.text main_prog.o
```

```
main_prog.o: file format elf64-x86-64
```

```
RELOCATION RECORDS FOR [.text]:
```

| OFFSET           | TYPE           | VALUE                               |
|------------------|----------------|-------------------------------------|
| 0000000000000009 | R_X86_64_32    | .rodata                             |
| 000000000000000e | R_X86_64_PLT32 | puts-0x0000000000000004             |
| 000000000000001f | R_X86_64_PC32  | global_var-0x0000000000000004       |
| 0000000000000026 | R_X86_64_32    | .rodata+0x0000000000000016          |
| 0000000000000030 | R_X86_64_PLT32 | printf-0x0000000000000004           |
| 0000000000000036 | R_X86_64_PC32  | global_var-0x0000000000000008       |
| 0000000000000040 | R_X86_64_PC32  | global_var-0x0000000000000004       |
| 0000000000000047 | R_X86_64_32    | .rodata+0x000000000000002e          |
| 0000000000000051 | R_X86_64_PLT32 | printf-0x0000000000000004           |
| 000000000000005b | R_X86_64_PLT32 | library_function-0x0000000000000004 |
| 0000000000000065 | R_X86_64_PLT32 | local_function-0x0000000000000004   |

- Детальніше ці релокації не розглядатимемо – вони схожі на попередні.
- Зауважимо хіба, що і локальна і глобальна функція тут використовує той самий тип релокацій.

- Символи та дані main\_prog.o:

```
$ nm -C main_prog.o
 U global_var
 U library_function
0000000000000000 T local_function
0000000000000015 T main
 U printf
 U puts
```

```
$ objdump -s --section=.data main_prog.o
```

```
main_prog.o: file format elf64-x86-64
```

```
$ objdump -s --section=.rodata main_prog.o
```

```
main_prog.o: file format elf64-x86-64
```

```
Contents of section .rodata:
```

```
0000 4c6f6361 6c206675 6e637469 6f6e2063 Local function c
0010 616c6c65 6400476c 6f62616c 2076616c alled.Global val
0020 75652062 65666f72 653a2025 6900476c ue before: %i.Gl
0030 6f62616c 2076616c 75652061 66746572 obal value after
0040 3a202569 00 : %i.
```

- Подивимося тепер на машинний код.

```

• $ objdump -M intel -S library.o
..... <skipped>
0000000000000000 <library_function>:
#include "file1.h"

int global_var; // Глобальна змінна

void library_function(int arg)
{
 0: f3 0f 1e fa endbr64
 4: 55 push rbp
 5: 48 89 e5 mov rbp,rsp
 8: 48 83 ec 10 sub rsp,0x10
 c: 89 7d fc mov DWORD PTR [rbp-0x4],edi
 printf("Argument: %i\n", arg); // Виклик зовнішньої ф-ції
 f: 8b 45 fc mov eax,DWORD PTR [rbp-0x4]
 12: 89 c6 mov esi,eax
 14: bf 00 00 00 00 mov edi,0x0 # Адреса стрічки
 19: b8 00 00 00 00 mov eax,0x0 # Нуль FP аргументів
 1e: e8 00 00 00 00 call 23 <library_function+0x23>

```

- Зверніть увагу на команди за зміщеннями 0x14 і 0x1E – всюди нульові зміщення<sup>74</sup>.
- Нагадаємо пов'язані релокації:

```

0000000000000015 R_X86_64_32 .rodata
000000000000001f R_X86_64_PLT32 printf-0x0000000000000004

```

Вони якраз на байт зсунуті – вказуючи на початок зміщень в машинних командах.

<sup>74</sup>За зміщенням 0x19 – просто нуль як безпосереднє значення.

```

• <skipped>
 // Звертання до глобальної змінної із того ж модуля,
 // де вона визначена:
 printf("Global var: %i\n", global_var);
 # Нижче -- звертання до змінної з цього ж модуля відносно RIP
23: 8b 05 00 00 00 00 mov eax,DWORD PTR [rip+0x0] # 29 <library_function+0x29>
29: 89 c6 mov esi,eax
2b: bf 00 00 00 00 mov edi,0x0 # Адреса стрічки
30: b8 00 00 00 00 mov eax,0x0 # Нуль FP аргументів
35: e8 00 00 00 00 call 3a <library_function+0x3a>
 global_var = arg;
3a: 8b 45 fc mov eax,DWORD PTR [rbp-0x4]
3d: 89 05 00 00 00 00 mov DWORD PTR [rip+0x0],eax # 43 <library_function+0x43>
}
43: 90 nop
44: c9 leave
45: c3 ret

```

- Нагадаємо собі релевантні релокації:

```

0000000000000025 R_X86_64_PC32 global_var-0x0000000000000004
000000000000002c R_X86_64_32 .rodata+0x000000000000000e
0000000000000036 R_X86_64_PLT32 printf-0x0000000000000004
000000000000003f R_X86_64_PC32 global_var-0x0000000000000004

```

- Релокації в межах одного модуля, відносно RIP – R\_X86\_64\_PC32.
- ◇ Релокація за адресами 0x25 і 0x3F зсунуті на два байти відносно початку відповідних команд – оскільки опкоди двобайтові.
- Розглянувши об'єктні файли, перейдемо до фінального виконавчого файлу.

- Дизасембл цілого файлу – занадто великий, подивимося на обрані функції:

```
int main() {
 40178a: f3 0f 1e fa endbr64
 40178e: 55 push rbp
 40178f: 48 89 e5 mov rbp, rsp
 // Звертання до глобальної змінної із іншого модуля:
 printf("Global value before: %i", global_var);
 401792: 8b 05 58 5b 0c 00 mov eax, DWORD PTR [rip+0xc5b58]
 # 0xc5b58 + 401798 = 4c72f0; 4c72f0 - address of <global_var>
 401798: 89 c6 mov esi, eax
 40179a: bf 1a 80 49 00 mov edi, 0x49801a # <=====
 40179f: b8 00 00 00 00 mov eax, 0x0
 4017a4: e8 a7 9e 00 00 call 40b650 <_IO_printf> # <=====
 global_var = 13;
 4017a9: c7 05 3d 5b 0c 00 0d mov DWORD PTR [rip+0xc5b3d], 0xd
 # 4c72f0 <global_var>
 4017b0: 00 00 00 printf("Global value after: %i", global_var);
 4017b3: 8b 05 37 5b 0c 00 mov eax, DWORD PTR [rip+0xc5b37]
 # 4c72f0 <global_var>
 4017b9: 89 c6 mov esi, eax
 4017bb: bf 32 80 49 00 mov edi, 0x498032 # <=====
 4017c0: b8 00 00 00 00 mov eax, 0x0
 4017c5: e8 86 9e 00 00 call 40b650 <_IO_printf> # <=====
}
```

- Адреси змінних – повністю визначені (зверніть увагу на опкоди).

- Адреси виклику функцій – повністю визначені.

```

void local_function(){
 401775: f3 0f 1e fa endbr64 # Команда захисту від Meltdown (AKC)
 skipped
int main() {
 skipped
 // Виклик функції із іншого модуля:
 library_function(42);
 4017ca: bf 2a 00 00 00 mov edi,0x2a
 4017cf: e8 11 00 00 00 call 4017e5 <library_function>

 // Виклик функції із цього модуля:
 local_function();
 4017d4: b8 00 00 00 00 mov eax,0x0
 4017d9: e8 97 ff ff ff call 401775 <local_function>

 return 0;
 4017de: b8 00 00 00 00 mov eax,0x0
}
 4017e3: 5d pop rbp
 4017e4: c3 ret

00000000004017e5 <library_function>:

void library_function(int arg)
{
 4017e5: f3 0f 1e fa endbr64
 4017e9: 55 push rbp

```

- void library\_function(int arg)

```

{
 4017e5: f3 0f 1e fa endbr64
 4017e9: 55 push rbp
 4017ea: 48 89 e5 mov rbp, rsp
 4017ed: 48 83 ec 10 sub rsp, 0x10
 4017f1: 89 7d fc mov DWORD PTR [rbp-0x4], edi
 printf("Argument: %i\n", arg); // Виклик зовнішньої ф-ції
 4017f4: 8b 45 fc mov eax, DWORD PTR [rbp-0x4]
 4017f7: 89 c6 mov esi, eax
 4017f9: bf 49 80 49 00 mov edi, 0x498049
 4017fe: b8 00 00 00 00 mov eax, 0x0
 401803: e8 48 9e 00 00 call 40b650 <_IO_printf>
 // Звертання до глобальної змінної із того ж модуля,
 // де вона визначена:
 printf("Global var: %i\n", global_var);
 401808: 8b 05 e2 5a 0c 00 mov eax, DWORD PTR [rip+0xc5ae2]
 # 4c72f0 <global_var>
 40180e: 89 c6 mov esi, eax
 401810: bf 57 80 49 00 mov edi, 0x498057
 401815: b8 00 00 00 00 mov eax, 0x0
 40181a: e8 31 9e 00 00 call 40b650 <_IO_printf>
 global_var = arg;
 40181f: 8b 45 fc mov eax, DWORD PTR [rbp-0x4]
 401822: 89 05 c8 5a 0c 00 mov DWORD PTR [rip+0xc5ac8], eax
 # 4c72f0 <global_var>
}
 401828: 90 nop
 401829: c9 leave
 40182a: c3 ret

```

- В підсумку – ми побачили, як виглядає статичне лінування, із прив'язкою до фіксованих адрес.
- Ремарка: що відбувається між точкою входу та викликом `main` – розглянемо пізніше.
- Варіанти `static PIE`, `static PIC`, `PIE`, `PIC` можна подивитися в директорії `linking_example`.
- Оскільки в прикладах відбуваються звертання і до локальних для модуля, і до зовнішніх функцій та змінних, перейдемо зразу до варіанту, де бібліотеку оформлену як динамічну бібліотеку, а сам виконавчий файл є `PIC`.
- Нам знадобляться наступні поняття:
  - ◇ `PLT` – Procedure Linkage Table.
  - ◇ `GOT` – Global Offsets Table
  - ◇ Ці структури використовуються для динамічного зв'язування.
  - ◇ Потреба у них пов'язана із необхідністю мати можливість повторно використовувати той же сегмент коду для різних копій однієї програми – реалокатії здійснюються в `PLT/GOT`, і сам код ~~можна залишати незмінним~~<sup>75</sup>.

<sup>75</sup> Як контр-приклад, `Windows 3.xx`, за певних умов – наявність кількох сегментних даних, реалокатії виконував безпосередньо в коді, тому неможливо було запустити кілька копій однієї такої програми, оскільки, економлячи крихітні тогочасні ресурси, `Windows` завжди використовував єдиний сегмент коду для всіх запущених копій програми.

# Shared library and PIC executable

- Компілюємо:

```
gcc -c -O0 -g -fpic main_prog.c -o main_prog.o
gcc -c -O0 -g -fpic library.c -o library.o
gcc -O0 -g -fpic -shared -o liblibrary.so library.o
gcc -O0 -g -fpic main_prog.o liblibrary.so -o pic_so_main_prog \
 liblibrary.so
```

- Створені файли:

```
$ ls -l pic_so_main_prog *.o -X -r | tr -s ' ' | cut -d" " -f 5,9
4728 main_prog.o
4256 library.o
17544 pic_so_main_prog
```

## # Sizes:

| text | data | bss | dec  | hex | filename         |
|------|------|-----|------|-----|------------------|
| 204  | 0    | 4   | 208  | d0  | library.o        |
| 325  | 0    | 0   | 325  | 145 | main_prog.o      |
| 1303 | 544  | 8   | 1855 | 73f | liblibrary.so    |
| 1840 | 640  | 8   | 2488 | 9b8 | pic_so_main_prog |

```
$ ldd pic_so_main_prog
linux-vdso.so.1 (0x00007ffcd9bbe9000)
liblibrary.so => ./liblibrary.so (0x00007f45ae969000)
libc.so.6 => /lib/x86_64-linux-gnu/libc.so.6 (0x00007f4d89bcf000)
/lib64/ld-linux-x86-64.so.2 (0x00007f4d89e10000)

$ ldd liblibrary.so
linux-vdso.so.1 (0x00007ffe58598000)
libc.so.6 => /lib/x86_64-linux-gnu/libc.so.6 (0x00007f5ee9f70000)
/lib64/ld-linux-x86-64.so.2 (0x00007f5eeal1b1000)
```

- Детально аналізувати об'єктні файли не будемо, лише порівняємо способи звертання:

```

36: void library_function(int arg)
37: {
38: 0: f3 0f 1e fa endbr64
39: 4: 55 push rbp
40: 5: 48 89 e5 mov rbp, rsp
41: 8: 48 83 ec 10 sub rsp, 0x10
42: c: 89 7d fc mov DWORD PTR [rbp-0x4], edi
43: printf("Argument: %i\n", arg); // Виллик зовнішньої ф-ції
44: f: 8b 45 fc mov eax, DWORD PTR [rbp-0x4]
45: 12: 89 c6 mov esi, eax
46: 14: 48 8d 05 00 00 00 lea rax, [rip+0x0] # 1b <
47: 1b: 48 89 c7 mov rdi, rax
48: 1e: b8 00 00 00 00 mov eax, 0x0
49: 23: e8 00 00 00 00 call 28 <library_function+0x28>
50: // Звертання до глобальної змінної із того ж модуля,
51: // де вона визначена:
52: printf("Global var: %i\n", global_var);
53: 20: 48 8b 05 00 00 00 00 mov rax, QWORD PTR [rip+0x0]
54: 2f: 8b 00 mov eax, DWORD PTR [rax]
55: 31: 89 c6 mov esi, eax
56: 33: 48 8d 05 00 00 00 00 lea rax, [rip+0x0] # 3a <
57: 3a: 48 89 c7 mov rdi, rax
58: 3d: b8 00 00 00 00 mov eax, 0x0
59: 42: e8 00 00 00 00 call 47 <library_function+0x47>
60: global_var = arg;
61: 47: 48 8b 05 00 00 00 00 mov rax, QWORD PTR [rip+0x0]
62: 4e: 8b 55 fc mov edx, DWORD PTR [rbp-0x4]
63: 51: 89 10 mov DWORD PTR [rax], edx
64: }
65: 53: 90 nop
66: 54: c9 leave
67: 55: c3 ret

```

```

36: void library_function(int arg)
37: {
38: 0: f3 0f 1e fa endbr64
39: 4: 55 push rbp
40: 5: 48 89 e5 mov rbp, rsp
41: 8: 48 83 ec 10 sub rsp, 0x10
42: c: 89 7d fc mov DWORD PTR [rbp-0x4], edi
43: printf("Argument: %i\n", arg); // Виллик зовнішньої ф-ції
44: f: 8b 45 fc mov eax, DWORD PTR [rbp-0x4]
45: 12: 89 c6 mov esi, eax
46: 14: bf 00 00 00 00 mov edi, 0x0
47: 19: b8 00 00 00 00 mov eax, 0x0
48: 1e: e8 00 00 00 00 call 23 <library_function+0x23>
49: // Звертання до глобальної змінної із того ж модуля,
50: // де вона визначена:
51: printf("Global var: %i\n", global_var);
52: 23: 8b 05 00 00 00 00 00 mov rax, QWORD PTR [rip+0x0]
53: 29: 89 c6 mov esi, eax
54: 2b: bf 00 00 00 00 mov edi, 0x0
55: 30: b8 00 00 00 00 mov eax, 0x0
56: 35: e8 00 00 00 00 call 3a <library_function+0x3a>
57: global_var = arg;
58: 3a: 8b 45 fc mov eax, DWORD PTR [rbp-0x4]
59: 3d: 89 05 00 00 00 00 00 mov DWORD PTR [rip+0x0], eax
60: }
61: 43: 90 nop
62: 44: c9 leave
63: 45: c3 ret

```

- Див. рядки 46-47 – в межах модуля адреса генерується відносно RIP, але відповідна reloкація є типу R\_X86\_64\_PC32, що вже траплявся раніше.
- Рядки 53-55 показують до глобальної змінної з цієї бібліотеки – відносно RIP береться адреса, звідки і візьметься адреса змінної.
- Така reloкація має тип R\_X86\_64\_REX\_GOTPCRELX.
- А рядок 54 – читає адресу із GOT.

- Також, якщо глянути в main\_prog.o, то виклик library\_function() відбувається звичним чином, хіба що реалокція типу R\_X86\_64\_PLT32:

```
0000000000000073 R_X86_64_PLT32 library_function-0x0000000000000000
```

```
 library_function(42);
6d: bf 2a 00 00 00 mov edi,0x2a
72: e8 00 00 00 00 call 77 <main+0x5d>
```

## Код liblibrary.so – звертання до даних

- Подивимося, як liblibrary.so звертається до глобальної змінної:

```
global_var = arg;
1160: 48 8b 05 79 2e 00 00 mov rax,QWORD PTR [rip+0x2e79]
 # 3fe0 <global_var@@Base-0x4c>
1167: 8b 55 fc mov edx,DWORD PTR [rbp-0x4] # arg
116a: 89 10 mov DWORD PTR [rax],edx
```

- [rbp-0x4] – адреса arg, тут нічого особливого.
- Команда, розташована за зміщенням 1160, отримує з адреси 0x3FE0 (утвореної відносно RIP) адресу змінної global\_var, зберігає її в RAX. Потім ця адреса використовується командою за зміщенням 0x116A для збереження в global\_var.
- Подивимося, якій секції це відповідає:

```
objdump -h pic_so_main_prog
..... skipped
23 .got 00000060 00000000000003fa0 00000000000003fa0 00002fa0 2**3
 CONTENTS, ALLOC, LOAD, DATA
24 .data 00000010 00000000000004000 00000000000004000 00003000 2**3
 CONTENTS, ALLOC, LOAD, DATA
```

- Тобто, 0x3FE0 – запис в GOT.
- Природно, що адреса глобальної змінної – в таблиці глобальних об'єктів.

## Код виконавчого файлу – виклик функцій з того ж модуля

- Оскільки звертання до глобальних експортованих змінних з .so і з програми збігається, подивимося на виклики функцій:

```
void local_function(){
 1189: f3 0f 1e fa endbr64
 118d: 55 push rbp

 // Виклик функції із цього модуля:
 local_function();
 1200: b8 00 00 00 00 mov eax,0x0
 1205: e8 7f ff ff ff call 1189 <local_function>
```

- Тобто, локальний виклик відбувається відносно RIP, але без іншої магії.

# Код виконавчого файлу – виклик функцій з `liblibrary.so`

```
0000000000001090 <library_function@plt>:
 1090: f3 0f 1e fa endbr64
 1094: f2 ff 25 2d 2f 00 00 bnd jmp QWORD PTR [rip+0x2f2d] # 3fc8 <library_function@plt>

 // Виклик функції із іншого модуля:
 library_function(42);
 11f6: bf 2a 00 00 00 mov edi,0x2a
 11fb: e8 90 fe ff ff call 1090 <library_function@plt>
```

Виклик імпортованої функції – через заглушку в PLT (за адресою 1090), яка бере адресу реальної функції із GOT.

- `bnd` – префікс, пов'язаний із захистом, тут його можна знехтувати, як і `endbr64`.
- Для повноти:

```
objdump -h pic_so_main_prog
..... skipped
12 .plt 00000040 0000000000001020 0000000000001020 00001020 2**4
 CONTENTS, ALLOC, LOAD, READONLY, CODE
13 .plt.got 00000010 0000000000001060 0000000000001060 00001060 2**4
 CONTENTS, ALLOC, LOAD, READONLY, CODE
14 .plt.sec 00000030 0000000000001070 0000000000001070 00001070 2**4
 CONTENTS, ALLOC, LOAD, READONLY, CODE
15 .text 00000171 00000000000010a0 00000000000010a0 000010a0 2**4
 CONTENTS, ALLOC, LOAD, READONLY, CODE
```

- Наявність кількох GOT і PLT таблиць має технічні причини, див., наприклад, <https://stackoverflow.com/questions/60371246/why-does-linux-use-two-got-sections-in-x64-got-vs-got-plt>.

- Базова адреса цього виконавчого файлу 0x10A0:

```
$ objdump -f pic_so_main_prog
```

```
pic_so_main_prog: file format elf64-x86-64
architecture: i386:x86-64, flags 0x00000150:
HAS_SYMS, DYNAMIC, D_PAGED
start address 0x000000000000010a0
```

- Реальна адреса визначається під час завантаження (приклад запуску з-під GDB):

```
Breakpoint 1, main () at main_prog.c:12
12 printf("Global value before: %i", global_var);
=> 0x000055555555551b <main+8>: 48 8b 05 2e 2e 00 00 mov rax,QWORD PTR [rip+0x2e2e]
0x000055555555551b2 <main+15>: 8b 00 mov eax,DWORD PTR [rax]
0x000055555555551b4 <main+17>: 89 c6 mov esi,eax
0x000055555555551b6 <main+19>: 48 8d 05 5d 0e 00 00 lea rax,[rip+0xe5d]
 # 0x55555555601a
```

- Якщо включено опцію динамічного завантаження `BIND_NOW`<sup>76</sup> – створенням однойменної змінної або при компіляції, за допомогою опції `-Wl,-z,now`, динамічне лінування відбудеться при завантаженні – відповідна GOT вже міститиме адреси всіх динамічних функцій.

◇ Це захищає від атак перезапису GOT – остання буде read-only<sup>77</sup>.

◇ Побачити роботу динамічного завантажувача можна якимось так:

```
LD_DEBUG=all LD_LIBRARY_PATH=. ./pic_so_main_prog
```

а наявність прапорця:

```
$ readelf -a pic_so_main_prog | grep BIND
0x000000000000001e (FLAGS) BIND_NOW
```

<sup>76</sup>В Ubuntu ввімкнено з версії 16.

<sup>77</sup><https://www.redhat.com/en/blog/hardening-elf-binaries-using-relocation-read-only-relro> та

<https://hockeyinjune.medium.com/relro-relocation-read-only-c8d0933faef3>.

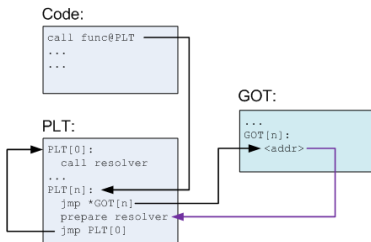
- Якщо ж `BIND_NOW` не задано (`-Wl,-z,lazy`), за першого стрибка до:

```
00000000000001090 <library_function@plt>:
1090: f3 0f 1e fa endbr64
1094: f2 ff 25 2d 2f 00 00 bnd jmp QWORD PTR [rip+0x2f2d]
 # 3fc8 <library_function@Base>
```

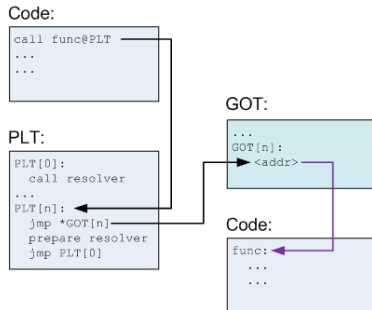
за адресою `0x3FC8` буде код динамічного лінування – який відпрацює та замінить свою адресу на реальну адресу функції.

- Цей процес гарно описують ілюстрації з статті <https://eli.thegreenplace.net/2011/11/03/position-independent-code-pic-in-shared-libraries>, хоча подробиці коду, створеного сучасним GCC, відрізняються:

### Перший виклик



### Всі наступні



- Познайомившись із тим, як компілятор створює релокації, PIC код та здійснює динамічне лінування, повернемося до асемблера.
- Приклад — `hello_linux64_libc.s`:

```
.intel_syntax noprefix # Same as: -msyntax=intel -mnaked-reg
.global main

.text

main:
 push rbp # Stack should be aligned to 16!

 call getpid
 mov esi, eax # Результат виклику -- в EAX
 lea rdi, [RIP+msg] # Lea а не mov, оскільки це лаконічнішн.
 xor rax, rax # Кількість використаних XMM регістрів для variadic arg
 call printf # "printf rdi, rsi"

 xor edi, edi # exit code
 call exit # "exit (0)" ==> "exit edi"

.data
msg: .asciz "Current process ID is %d.\n"

Еквівалентно до: printf(msg, getpid());
```

- Зверніть увагу на нулі-заглушки – як і в коді, створеному GCC.
- Також, адресація йде відносно RIP.

```
$ objdump -M intel -d hello_linux64_libc.o
hello_linux64_libc.o: file format elf64-x86-64
```

Disassembly of section .text:

```
0000000000000000 <main>:
 0: 55 push rbp
 1: e8 00 00 00 00 call 6 <main+0x6> ## getpid
 6: 89 c6 mov esi,eax
 8: 48 8d 3d 00 00 00 00 lea rdi,[rip+0x0] # f <main+0xf>
 f: 48 31 c0 xor rax,rax
12: e8 00 00 00 00 call 17 <main+0x17> ## printf
17: 31 ff xor edi,edi
19: e8 00 00 00 00 call 1e <main+0x1e> ## exit
```

- Формат очевидний – ліворуч зміщення початку інструкції в байтах, далі двійкове представлення, далі – мнемоніка.
- Коментарі після '##' – мої.

```
hello_linux64_libc.o: file format elf64-x86-64
hello_linux64_libc.o
architecture: i386:x86-64, flags 0x00000011:
HAS_RELOC, HAS_SYMS
start address 0x0000000000000000
```

#### Sections:

| Idx | Name                                         | Size     | VMA              | LMA              | File off | Algn |
|-----|----------------------------------------------|----------|------------------|------------------|----------|------|
| 0   | .text                                        | 0000001e | 0000000000000000 | 0000000000000000 | 00000040 | 2**0 |
|     | CONTENTS, ALLOC, LOAD, RELOC, READONLY, CODE |          |                  |                  |          |      |
| 1   | .data                                        | 0000001b | 0000000000000000 | 0000000000000000 | 0000005e | 2**0 |
|     | CONTENTS, ALLOC, LOAD, DATA                  |          |                  |                  |          |      |
| 2   | .bss                                         | 00000000 | 0000000000000000 | 0000000000000000 | 00000079 | 2**0 |
|     | ALLOC                                        |          |                  |                  |          |      |

#### SYMBOL TABLE:

| Address          | Size | Type  | Symbol | File                    |
|------------------|------|-------|--------|-------------------------|
| 0000000000000000 | 1    | d     | .data  | 0000000000000000 .data  |
| 0000000000000000 | 1    |       | .data  | 0000000000000000 msg    |
| 0000000000000000 | g    |       | .text  | 0000000000000000 main   |
| 0000000000000000 |      | *UND* |        | 0000000000000000 getpid |
| 0000000000000000 |      | *UND* |        | 0000000000000000 printf |
| 0000000000000000 |      | *UND* |        | 0000000000000000 exit   |

#### RELOCATION RECORDS FOR [.text]:

| OFFSET           | TYPE           | VALUE                     |
|------------------|----------------|---------------------------|
| 0000000000000002 | R_X86_64_PLT32 | getpid-0x0000000000000004 |
| 000000000000000b | R_X86_64_PC32  | .data-0x0000000000000004  |
| 0000000000000013 | R_X86_64_PLT32 | printf-0x0000000000000004 |
| 000000000000001a | R_X86_64_PLT32 | exit-0x0000000000000004   |

- Окремі частини цієї інформації можна отримати за допомогою відповідних ключів objdump: "-h" – секції, "-t" – таблицю символів, "-r" – релокації тощо.

## Лінкуємо – static, no-PIE

- Останнім часом GCC в популярних дистрибутивах часто налаштовано компілювати як PIE за замовчуванням.
- Тому, для гарантії, вимкнемо примусово. Почнемо із статичного варіанту:

```
gcc hello_linux64_libc.o -o hello_linux64_libc -static
```

- -no-pie тут давати можна, але на результат воно не впливає. Для платформ, де статичний виконавчий файл може бути релоковано, є також опція "-static-pie", "наш" GCC її не знає.
- Зверніть увагу, -no-pie – опція для лінкера, а -fno-pie, -fpie, -fno-pic, -fpic – для компілятора, тут вони зайві і ні на що не впливатимуть.
- В більш широкій перспективі див. також цю відповідь: <https://stackoverflow.com/a/46493456>.
- Варто після цього подивитися на результат:

```
objdump -x -M intel hello_linux64_libc > hello_linux64_libc.exe.header
objdump -d -M intel hello_linux64_libc > hello_linux64_libc.exe.disasm
readelf -h hello_linux64_libc > hello_linux64_libc.exe.readelf
```

Відповідні файли будуть достатньо великі, тому нижче розглянемо лише ключове.

# Readelf result:

## ELF Header:

```
Magic: 7f 45 4c 46 02 01 01 03 00 00 00 00 00 00 00 00
Class: ELF64
Data: 2's complement, little endian
Version: 1 (current)
OS/ABI: UNIX - GNU
ABI Version: 0
Type: EXEC (Executable file) <--- v--
Machine: Advanced Micro Devices X86-64
Version: 0x1
Entry point address: 0x4016a0 <=====
Start of program headers: 64 (bytes into file)
Start of section headers: 783576 (bytes into file)
Flags: 0x0
Size of this header: 64 (bytes)
Size of program headers: 56 (bytes)
Number of program headers: 10
Size of section headers: 64 (bytes)
Number of section headers: 28
Section header string table index: 27
```

- Стрілки – мої.

# Дизасемлювання виконавчого файлу

- Результат – здоровенний. Чого саме там так багато – поговоримо трішки пізніше.
- Поки, глянемо на, власне, нашу програму:

00000000004017c5 <main>:

|         |                      |      |                                  |
|---------|----------------------|------|----------------------------------|
| 4017c5: | 55                   | push | rbp                              |
| 4017c6: | e8 55 69 01 00       | call | 418120 <__getpid>                |
| 4017cb: | 89 c6                | mov  | esi,eax                          |
| 4017cd: | 48 8d 3d fc 88 0a 00 | lea  | rdi,[rip+0xa88fc] # 4aa0d0 <msg> |
| 4017d4: | 48 31 c0             | xor  | rax,rax                          |
| 4017d7: | e8 64 34 00 00       | call | 404c40 <_IO_printf>              |
| 4017dc: | 31 ff                | xor  | edi,edi                          |
| 4017de: | e8 dd 31 00 00       | call | 4049c0 <exit>                    |

## Спробуємо запустити

```
$ gdb ./hello linux64 libc
```

<Купа тексту, ближче до кінця можуть бути повідомлення про помилку, не пропустіть>

```
(qdb) set disassembly-flavor intel
```

```
(qdb) break main
```

Breakpoint 1 at 0x4017c5

```
(qdb) run
```

```
Starting program: <....>/hello linux64 libc
```

Downloading separate debug info for system-supplied DSO at 0x7ffff7ffd000

Breakpoint 1, 0x00000000004017c5 in main ()

```
(gdb) disas
```

Dump of assembler code for function main:

```
=> 0x0000000000004017c5 <+0>: push rbp
0x0000000000004017c6 <+1>: call 0x418120 <getpid>
0x0000000000004017cb <+6>: mov esi,eax
0x0000000000004017cd <+8>: lea rdi,[rip+0xa88fc] # 0x4aa0d0
0x0000000000004017d4 <+15>: xor rax,rax
0x0000000000004017d7 <+18>: call 0x404c40 <printf>
0x0000000000004017dc <+23>: xor edi,edi
0x0000000000004017de <+25>: call 0x4049c0 <exit>
0x0000000000004017e3 <+30>: cs nop WORD PTR [rax+rax*1+0x0]
0x0000000000004017ed <+40>: nop DWORD PTR [rax]
```

End of assembler dump.

```
(gdb) x /s 0xa88fc+0x4017d4 <Є ідеї, звідки взялося 0x4017d4?>
```

```
0x4aa0d0: "Current process ID is %d.\n"
```

# Лінкуємо – no-PIE

- Лінуємо<sup>78</sup>:

```
gcc hello_linux64_libc.o -o hello_linux64_libc -no-pie
```

- Розмір – 15960.
- Повторюємо:

```
objdump -x -M intel hello_linux64_libc > hello_linux64_libc.exe.header
objdump -d -M intel hello_linux64_libc > hello_linux64_libc.exe.disasm
readelf -h hello_linux64_libc > hello_linux64_libc.exe.readelf
```

Отримані на цьому етапі файли теж помітно менші.

---

<sup>78</sup>За замовчуванням використовуватиме shared-бібліотеки, тому відповідна опція тут не потрібна. ▶ ◀ ≡ ≡ ≡

# Readelf result:

## ELF Header:

```

Magic: 7f 45 4c 46 02 01 01 00 00 00 00 00 00 00 00 00
Class: ELF64
Data: 2's complement, little endian
Version: 1 (current)
OS/ABI: UNIX - System V <---
ABI Version: 0
Type: EXEC (Executable file)
Machine: Advanced Micro Devices X86-64
Version: 0x1
Entry point address: 0x401060 <=====
Start of program headers: 64 (bytes into file)
Start of section headers: 14040 (bytes into file)
Flags: 0x0
Size of this header: 64 (bytes)
Size of program headers: 56 (bytes)
Number of program headers: 13
Size of section headers: 64 (bytes)
Number of section headers: 30
Section header string table index: 29
```

- Стрілки – мої.

# Дизасемлювання виконавчого файлу

Disassembly of section .plt:

```
0000000000401020 <getpid@plt-0x10>: # Коментар збиває з пантелику - це просто перед getpid@plt
 401020: push QWORD PTR [rip+0x2fca] # 403ff0 <_GLOBAL_OFFSET_TABLE_+0x8>
 401026: jmp QWORD PTR [rip+0x2fcc] # 403ff8 <_GLOBAL_OFFSET_TABLE_+0x10>
 40102c: nop DWORD PTR [rax+0x0]
```

0000000000401030 <getpid@plt>:

```
 401030: jmp QWORD PTR [rip+0x2fca] # 404000 <getpid@GLIBC_2.2.5>
 401036: push 0x0
 40103b: jmp 401020 <_init+0x20>
```

0000000000401040 <printf@plt>:

```
 401040: jmp QWORD PTR [rip+0x2fc2] # 404008 <printf@GLIBC_2.2.5>
 401046: push 0x1
 40104b: jmp 401020 <_init+0x20>
```

0000000000401050 <exit@plt>:

```
 401050: jmp QWORD PTR [rip+0x2fba] # 404010 <exit@GLIBC_2.2.5>
 401056: push 0x2
 40105b: jmp 401020 <_init+0x20>
```

<----- skipped ----->

0000000000401146 <main>:

```
 401146: 55 push rbp
 401147: e8 e4 fe ff ff call 401030 <getpid@plt>
 40114c: 89 c6 mov esi,eax
 40114e: 48 8d 3d d3 2e 00 00 lea rdi,[rip+0x2ed3] # 404028 <msg>
 401155: 48 31 c0 xor rax,rax
 401158: e8 e3 fe ff ff call 401040 <printf@plt>
 40115d: 31 ff xor edi,edi
 40115f: e8 ec fe ff ff call 401050 <exit@plt>
```

# Лінкуємо – PIE

- `gcc hello_linux64_libc.o -o hello_linux64_libc -pie`
- Повторюємо:  

```
objdump -x -M intel hello_linux64_libc > hello_linux64_libc.exe.header
objdump -d -M intel hello_linux64_libc > hello_linux64_libc.exe.disasm
readelf -h hello_linux64_libc > hello_linux64_libc.exe.readelf
```
- Розмір – 16040.
- Зауважте – файл став трохи більшим, ніж був у no-pie випадку – незалежність від позиції не дається безкоштовно. Подивимося, як саме доводиться платити.

## ELF Header:

```
Magic: 7f 45 4c 46 02 01 01 00 00 00 00 00 00 00 00
Class: ELF64
Data: 2's complement, little endian
Version: 1 (current)
OS/ABI: UNIX - System V
ABI Version: 0 v-----
Type: DYN (Position-Independent Executable file)
Machine: Advanced Micro Devices X86-64
Version: 0x1
Entry point address: 0x1070 <=====
Start of program headers: 64 (bytes into file)
Start of section headers: 14120 (bytes into file)
Flags: 0x0
Size of this header: 64 (bytes)
Size of program headers: 56 (bytes)
Number of program headers: 13
Size of section headers: 64 (bytes)
Number of section headers: 30
Section header string table index: 29
```

- Стрілки – мої.

# Дизасемлювання виконавчого файлу

Disassembly of section .plt:

```
0000000000001020 <getpid@plt-0x10>: # Коментар збиває з пантелику - це просто перед getpid@plt
1020: push QWORD PTR [rip+0x2f8a] # 3fb0 <_GLOBAL_OFFSET_TABLE_+0x8>
1026: jmp QWORD PTR [rip+0x2f8c] # 3fb8 <_GLOBAL_OFFSET_TABLE_+0x10>
102c: nop DWORD PTR [rax+0x0]
```

```
0000000000001030 <getpid@plt>:
1030: jmp QWORD PTR [rip+0x2f8a] # 3fc0 <getpid@GLIBC_2.2.5>
1036: push 0x0
103b: jmp 1020 <_init+0x20> <=====
```

```
0000000000001040 <printf@plt>:
1040: jmp QWORD PTR [rip+0x2f82] # 3fc8 <printf@GLIBC_2.2.5>
1046: push 0x1
104b: jmp 1020 <_init+0x20>
```

```
0000000000001050 <exit@plt>:
1050: jmp QWORD PTR [rip+0x2f7a] # 3fd0 <exit@GLIBC_2.2.5>
1056: push 0x2
105b: jmp 1020 <_init+0x20>
```

<----- skipped ----->

```
0000000000001159 <main>:
1159: 55 push rbp
115a: e8 d1 fe ff ff call 1030 <getpid@plt> <=====
115f: 89 c6 mov esi,eax
1161: 48 8d 3d a8 2e 00 00 lea rdi,[rip+0x2ea8] # 4010 <msg>
1168: 48 31 c0 xor rax,rax
116b: e8 d0 fe ff ff call 1040 <printf@plt>
1170: 31 ff xor edi,edi
```

```
(gdb) set disassembly-flavor intel
(gdb) b main
Breakpoint 1 at 0x1159
(gdb) r
Starting program: <...>/hello_linux64_libc
Breakpoint 1, 0x000055555555159 in main ()
(gdb) disas
Dump of assembler code for function main:
=> 0x000055555555159 <+0>: push rbp
 0x00005555555515a <+1>: call 0x55555555030 <getpid@plt>
 0x00005555555515f <+6>: mov esi,eax
 0x000055555555161 <+8>: lea rdi,[rip+0x2ea8] # 0x555555558010
 0x000055555555168 <+15>: xor rax,rax
 0x00005555555516b <+18>: call 0x55555555040 <printf@plt>
 0x000055555555170 <+23>: xor edi,edi
 0x000055555555172 <+25>: call 0x55555555050 <exit@plt>
End of assembler dump.
```

- Є версія, за якою базовою адресою було завантажено програму?

## Щодо 32-бітового коду

- 32-бітовий код адресувати відносно RIP не може. Тому мусить застосовувати наступний трюк:

```

 call @f
@@:
 pop ebx
 sub ebx, rva @b ; @b - rva, де @b -- адреса в GOT

```

- При виклику адреса повернення збережеться в стеку, ми її звідти добуваємо.
- Часто так робити не варто – це дуже збиває з пантелику сучасні процесори (їх *call/return predictor*), але раз на запуск програми – не зробить великої шкоди. (Кращий варіант – далі).
- Оператор `fasm rva`, перетворює адресу у відносну до GOT. Такі адреси можна використовувати в PIC-коді.
- Ще один, "суміжний" оператор `fasm` – `plt`, виклик підпрограми через `plt`:

```

 call PLT printf

```

- `@@` – локальна мітка, перехід чи виклик `@f` означає перейти на найближчу мітку вперед, `@b` – назад.
- (В архіві із прикладами є директорії `pie_test32` та `pie_test64`, із всіма файлами, про які йшла мова вище).

- Як фокус вище виглядає у реальному коді? Якщо глибоко залізти в дизасемблерний лістинг коду для 32-бітного Лінукса із використанням `libc`, можна знайти таке:

```
08048370 <_start>: ; Точка входу у виконавчий файл
08048370: xor ebp,ebp
08048372: pop esi
08048373: mov ecx,esp
08048375: and esp,0xffffffff0
08048378: push eax
08048379: push esp
0804837a: push edx
0804837b: call 80483a3 <_start+0x33> <-- 1
08048380: add ebx,0x1c80 <-- 3
08048386: lea eax,[ebx-0x1af0]
0804838c: push eax
0804838d: lea eax,[ebx-0x1b50]
08048393: push eax
08048394: push ecx
08048395: push esi
08048396: mov eax,0x8048488
0804839c: push eax
0804839d: call 8048350 <__libc_start_main@plt> ;
080483a2: hlt
080483a3: mov ebx,DWORD PTR [esp] <-- 2, save EIP
080483a6: ret <-- return to 3
```

- `__libc_start_main` – функція в глибині `glibc`, яка готує необхідне середовище та викликає вашу `main()` як звичайну функцію.

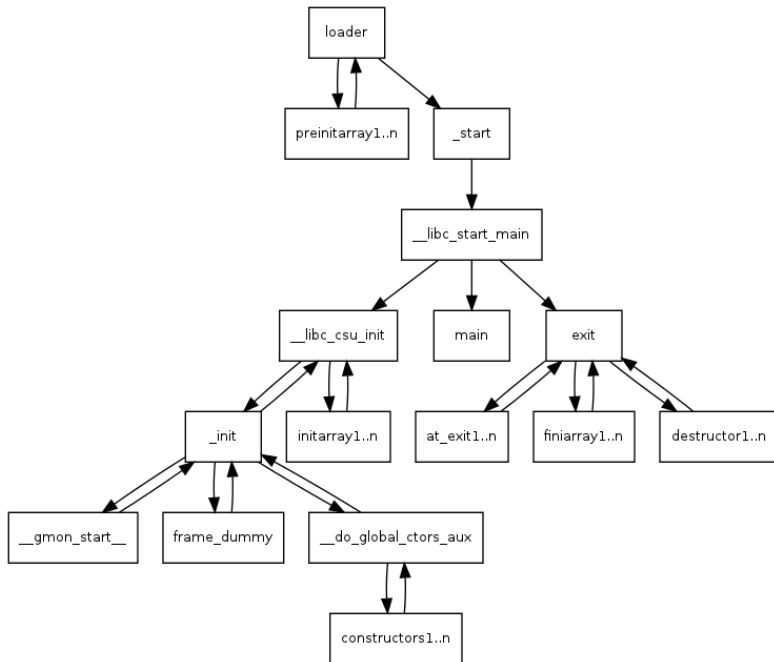
- Її прототип:

```
int __libc_start_main(int (*main) (int, char**, char**),
 int argc,
 char *__unbounded *__unbounded ubp_av,
 void (*init) (void),
 void (*fini) (void),
 void (*rtld_fini) (void),
 void (*__unbounded stack_end));
```

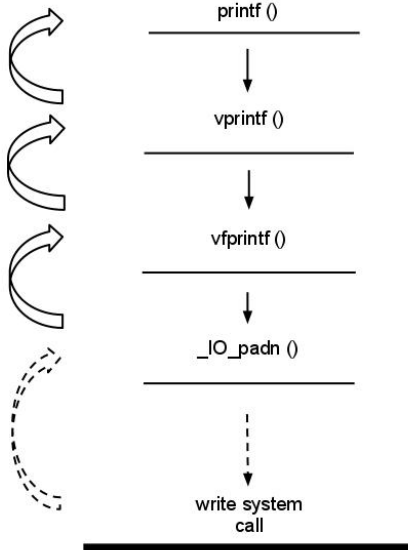
- В статичному варіанті вона буде безпосередньо в коді. Тоді виклик `main()` виглядатиме якимось так:

```
mov rdx,QWORD PTR [rip+0x2bacef] # 6bbdc8 <__environ>
mov rsi,QWORD PTR [rsp+0x10]
mov edi,DWORD PTR [rsp+0xc]
mov rax,QWORD PTR [rsp+0x18]
call rax <-- адреса main()
mov edi,eax
call 40ea30 <exit>
```

- `_start` – власне, точка входу у наш виконавчий файл. Для чисто асемблерних програм, там все і почнеться.
- Подробиці ми зараз не розглядатимемо, див.  
<http://dbp-consulting.com/tutorials/debugging/linuxProgramStartup.html>, схема звідки – на наступному слайді.
- Чисто практичні наслідки такого підходу для Embedded-систем розглядалися тут: <http://indrekis2.blogspot.com/2015/11/c-arm-gcc-stm32-coide.html>



# Ще про "нутроці"



# Інтринсики (Intrinsic function)

- Програмувати на асемблері важко, а такий код – непортабельний.
- Крім того, багато можливостей відповідних платформ не доступні із стандартного C/C++.
- Компілятори надають певну допомогу у вирішенні цих проблем<sup>79</sup>.
- Робить він це за допомогою так званих інтринсик – спеціальних конструкцій, які виглядають як звичайні функції, однак, виконують спеціальні дії. (GCC також називає їх Built-in Functions).
- Зокрема, це може бути вставляння в код однієї чи більшої кількості спеціальних інструкцій процесора.
- Використовуються для виконання якихось системних дій, наприклад – перехід в режим сну AVR8:

```
void __builtin_avr_sleep (void);
```

вставляє інструкцію SLEEP відповідного MCU, а `_dmb(0xF);` (компілятор Keil) – вставляє інструкцію DMB – "Data Memory Barrier"<sup>80</sup>.

<sup>79</sup>В сенсі – на додачу до позбавляння нас необхідності писати на асемблері. :-)

<sup>80</sup>Бар'єри будемо більше обговорювати в курсі АКС.

- Інше важливе використання інтринсик – використання векторних розширень. Хоча сучасні компілятори намагаються здійснювати автовекторизацію, вона часто неефективна.
- Крім того, використання інтринсик трішки збільшує портабельність коду.
- Приклад: додавання двох double, вирівняних на границю 16 байт:

```
; SSE2, vector, aligned
 MOVAPD XMM0, [vect_double_1]
 MOVAPD XMM1, [vect_double_2]
 ADDPD XMM0, XMM1
 MOVAPD [vect_double_3], XMM0
```

- На C це може виглядати так:

```
#include <x86intrin.h>
```

```
.....
```

```
__v2df v1 = {1.2, 3.4};
__v2df v2 = {5.6, 7.8};
__v2df v3 = _mm_add_pd(v1, v2);
```

## Посилання

- Непогана оглядова презентація: [https://deplinenoise.files.wordpress.com/2015/03/gdc2015\\_afredriksson\\_simd.pdf](https://deplinenoise.files.wordpress.com/2015/03/gdc2015_afredriksson_simd.pdf).
- Інтринсики MSVC:  
<https://msdn.microsoft.com/en-us/library/26td2lds.aspx>.
- Інтринсики GCC:  
<https://gcc.gnu.org/onlinedocs/gcc/Target-Builtins.html> та суміжна тема, векторне розширення:  
<https://gcc.gnu.org/onlinedocs/gcc/Vector-Extensions.html>, див. також <https://www.linuxjournal.com/content/introduction-gcc-compiler-intrinsics-vector-processing>.
- Інтринсики Intel (GCC для відповідних платформ їх підтримує), інтерактивний довідник: <https://software.intel.com/sites/landingpage/IntrinsicsGuide/>.
- Ряд додаткових посилань:  
<https://stackoverflow.com/tags/sse/info>.
- Список хідерів по підмножинах інтринсик (хоча, `x86intrin.h` включає їх всіх): <https://stackoverflow.com/a/11228864>.
- Автоматична векторизація GCC: <https://www.gnu.org/software/gcc/projects/tree-ssa/vectorization.html>.

## Inline асемблер

- Ще одним способом не писати всю програму на асемблері, є inline-асемблер – можливість вставляти асемблерний код безпосередньо в C, C++, Pascal і т.д., програму.
- GCC має дуже розвинуту та гнучку підтримку вставляння асемблерного коду. На жаль, 64-бітовий MSVC не підтримує inline асемблер взагалі.
- Найпростіший приклад:

```
asm("nop");
```

- Вище – так звана базова асемблерна вставка, вона не особливо цікава. Але можна й так:

```
int src = 1;
int dst;
asm("mov EAX, %1\n"
 "mov %0, EAX\n"
 : "=r" (dst)
 : "r" (src)
 : "eax");
printf("%i\n", dst);
```

- Розглянемо її.

- GCC за замовчуванням використовує AT&T діалект асемблера. Щоб скомпільовалася асемблерна вставка вище, яка використовує діалект Intel, слід передати компілятору опцію `-masm=intel`:

```
gcc inline_asm.c -o inline_asm -masm=intel
```

- Вставка починається із команд асемблера. Зверніть увагу на `"\n"` в кінці кожної. Потім, після двокрапки – вихідні аргументи, потім вхідні аргументи та список регістрів, що можуть "псуватися" даною асемблерною вставкою:

```
int src = 1;
int dst;
asm("mov EAX, %1\n" // <== ASM line 1
 "mov %0, EAX\n" // <== ASM line 2
 : "=r" (dst) // <== Output, register
 : "r" (src) // <== Input, register
 : "eax"); // <== Clobber list
printf("%i\n", dst);
```

- `"=`" означає, що в аргумент відбувається лише запис. Якщо він використовується і для читання і для запису, слід використовувати `"+"`.

- "cc" в Clobber list – вставка псує прапорці (стосується практично будь-яких арифметичних команд x86).
- "memory" – вказує, що вставка здійснює операції із пам'яттю, додатково до звертань до своїх аргументів<sup>81</sup>.
- Компілятор має право переставляти асемблерні вставки. Якщо потрібно йому це заборонити, можна оголосити її як "asm volatile".
- Є можливість стрибати за межі, за допомогою "asm goto".
- Є можливість робити вставку сумісною із різними діалектами асемблера, наприклад, AT&T та Intel. Робиться це за допомогою фігурних дужок, у яких перераховано варіанти для кожного діалекту. Наприклад:

```
xchg{l} {%%}ebx, %1
```

перетвориться в:

```
xchgl %%ebx, %1 /* Компілюємо із діалектом AT&T */
```

або:

```
xchg ebx, %1 /* Компілюємо із діалектом Intel */
```

---

<sup>81</sup>Змушує компілятор вважати будь-яку пам'ять зміненою – служить бар'єром пам'яті для процесора, хоча і не для CPU – див. курс АКС. Заважає оптимізації, природно.

## Приклад для ARM Cortex M, ще придасться нам на практичних

```
__attribute__((always_inline)) static inline int
SH_DoCommand(int in_r0, int in_r1, int *out_r0)
{
 int res = 0;
 asm volatile (
 " mov r0, %[cmd] \n"
 " mov r1, %[arg] \n"
 " bkpt 0xAB \n"
 " mov %[res], r0 \n"

 : [res] "=r" (out_r0) /* Output */
 : [cmd] "r" (in_r0), [arg] "r" (in_r1) /* Inputs */
 : "r0", "r1", "cc", "memory"
);
 if(out_r0)
 *out_r0 = res;
}
return res;
}
```

- <http://indrekis2.blogspot.com/2017/12/semihosting.html>

## Посилання

- Детальніше див:
- <https://gcc.gnu.org/onlinedocs/gcc/Using-Assembly-Language-with-C.html>
- <https://www.ibiblio.org/gferg/ldp/GCC-Inline-Assembly-HOWTO.html> (достатньо старенький документ, 2003).
- Багато прикладів: [https://locklessinc.com/articles/gcc\\_asm/](https://locklessinc.com/articles/gcc_asm/)
- Правда, за всіма посиланнями вище використано діалект AT&T...
- Історичний приклад, inline-assembly в Open Watcom: <http://indrekis2.blogspot.com/2011/05/openwatcom.html>.
- Баг в CMSIS (ARM Cortex M), пов'язаний із ним: <http://indrekis2.blogspot.com/2013/04/arm-cmsis.html>.
- Кумедний баг: "[Inline Assembly Dangers](#)" – через пропущений символ нового рядка, цілий блок коду був сприйнятий як коментар.

## Діалекти асемблера AT&amp;T і Intel

|                             | Intel                                                                                     | AT&T                                                                          |
|-----------------------------|-------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------|
| Порядок операндів           | dest, src                                                                                 | src, dest                                                                     |
| Три операнди<br>Два src (!) | mov eax, 5<br>shld dest, src, cnt<br>enter 16, 0                                          | mov \$5, %eax<br>shld cnt, src, dest<br>enter 16,0                            |
| Розмір операндів            | Визначається автоматично, з імен регістрів<br>add esp, 4                                  | Суфікс у команді (b – byte, w – word, l – dword, q – qword)<br>addl \$4, %esp |
| Роль символу:               | Визначається автоматично                                                                  | \$ – безпосереднє значення, % – регістр (sigil).                              |
| Адреса                      | mov eax, [ebx + 4*ecx + disp]<br>За потреби, розмір може уточнюватися: dword [...] і т.д. | movl disp(%ebx,%ecx,4), %eax                                                  |
| Abs. jump<br>PC rel. jump   | jmp [label]<br>jmp label                                                                  | jmp *label<br>jmp label                                                       |

- Детальніше порівняння – див.  
[https://en.wikipedia.org/wiki/X86\\_assembly\\_language#Syntax](https://en.wikipedia.org/wiki/X86_assembly_language#Syntax).
- Порівняння GAS і NASM: <https://web.archive.org/web/20131003180256/http://www.ibm.com/developerworks/linux/library/l-gas-nasm/index.html>.

- В світі Windows домінує діалект Intel, в Unix-світі – AT&T, все ж, Юнікс з'явився в AT&T Bell Labs.
- Порядок операндів у них – зворотній. Крім випадку, коли мова про кілька джерел (на щастя, таких команд мало – `enter`, `bound` і системні).
- Це можна пережити, але, **УВАГА**, є жорстокий баг. Для підтримки зворотної сумісності із кодом, який генерував GCC і т.д., для некомутативних команд роботи із `floating point`, типу `fsub`, `fdiv`, свідомо генерується невірний, неочікуваний, код.
- ◇ Подобиці див:  
<http://www.mindfruit.co.uk/2012/03/trouble-with-fsub.html>.  
Особливо весело, що `objdump` іноді підігрує. Мінімалістична згадка про це в офіційній документації:  
[https://sourceware.org/binutils/docs/as/i386\\_002dBugs.html](https://sourceware.org/binutils/docs/as/i386_002dBugs.html).

## Посилання

- Офіційна документація GAS (GNU Assembler):  
<https://sourceware.org/binutils/docs/as/>. Зокрема, глава "AT&T Syntax versus Intel Syntax".
- <https://www.ibm.com/developerworks/library/l-gas-nasm/index.html>  
– "Linux assemblers: A comparison of GAS and NASM".
- [https://en.wikibooks.org/wiki/X86\\_Assembly/GAS\\_Syntax](https://en.wikibooks.org/wiki/X86_Assembly/GAS_Syntax).
- Про порядок для bound: <https://stackoverflow.com/a/52159249>.
- Про проблеми із fsub/fdiv:  
<http://www.mindfruit.co.uk/2012/03/trouble-with-fsub.html>.

## Посилання – інше

- [TIS-100](#) – гра в асемблер.

